

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF TECHNOLOGY
PANDIT DENDAYAL ENERGY UNIVERSITY
SESSION 2026-27

SUBMITTED BY

NAME	:	Patel Ishan
ROLL NO.	:	25BCP113
DIVISION	:	3
COURSE NAME	:	Data Structures
COURSE CODE	:	24CS203P

SUBMITTED TO

Dr. Hiren Thakkar

Department of Computer Science and Engineering

Pandit Deendayal Energy University

****

Pandit Deendayal Energy University

Raysan, Gandhinagar - 380 007, Gujarat, India

Computer Science and Engineering

Certificate

This is to certify that

Mr./Ms. **Patel Ishan** Roll no. **25BCP113**

of 3rd semester degree course in Computer Science and Engineering has satisfactorily completed his/her term work in **Data Structures Lab (24CS203P)** subject during the semester from **July 2026** to **Nov. 2026** at School of Technology, PDEU.

Date of Submission: _____

Signature:

Faculty In-charge Head of Department

INDEX

Exp. No.	Title of Lab Work	Date of Lab Work	Page	Signature
1	Revision of Arrays (Searching, Sorting & Extremum Elements)			
2	Revision of Structures (Student Records & Nested Organization-Employee)			
3	Revision of Pointers (Arrays of Pointers, Pointers to Arrays, Structures & Swapping)			
4	Linked List (Merge Sorted Lists, 3-Way Insertion/Deletion, Circular & Doubly LL)			
5	Stack (Array & Linked List Implementation, Recursive Stack Sorting)			
6	Stack Applications (Infix to Postfix, Postfix Evaluation & Parentheses Validation)			
7	Queue (Array Queue, Linked List Queue, Priority Queue & Deque Operations)			
8	Trees (BST Insert/Search/Delete, Tree Traversals & Preorder BST Construction)			
9	Graphs (Breadth First Search, Depth First Search, Cycle Detection & Kruskal's MST)			
10	Hashing (Direct Index Mapping Search in O(1) & Subset Verification)			

EXPERIMENT NO. 1

Revision of Arrays (Searching, Sorting & Extremum Elements)

1. Objective

To design, implement, and analyze fundamental array algorithms in C, including Linear Search, Binary Search, Bubble Sort, Insertion Sort, Selection Sort, and single-pass extremum tracking (minimum, maximum, second smallest, and second largest elements).

2. Problem Statements / Program Definitions

1. Write a program in C to perform linear and binary search.

2. Write a program in C to perform bubble sort, insertion sort and selection sort. Take the array size and array elements from user.

3. Write a program in C that obtains the minimum and maximum element from the array. Modify this program to give the second largest and second smallest element of the array.

3. Implementation & Output Snapshots

Program 1.1: Linear Search and Binary Search (E1Q1.c)

```
#include <stdio.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}

int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
```

```
            }
        }
    }
}

int main() {
    printHeader();

    int n, target;
    printf("Enter number of elements: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        printf("  Element [%d]: ", i + 1);
        scanf("%d", &arr[i]);
    }

    printf("\nEnter target element to search: ");
    scanf("%d", &target);

    // Linear Search
    printf("\n--- Performing Linear Search ---\n");
    int linIdx = linearSearch(arr, n, target);
    if (linIdx != -1) {
        printf("  [Linear Search] Element %d found at index %d (position %d).\n", target,
linIdx, linIdx + 1);
    } else {
        printf("  [Linear Search] Element %d not found in the array.\n", target);
    }

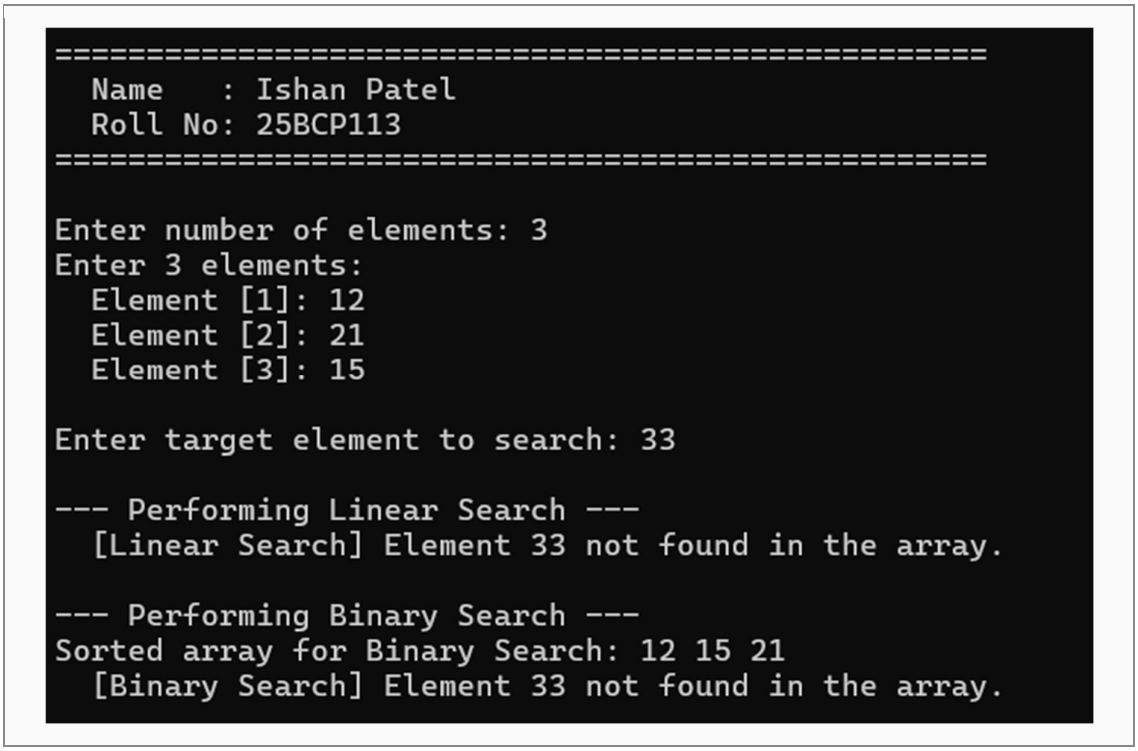
    // Binary Search requires sorted array
    int sortedArr[n];
    for (int i = 0; i < n; i++) {
        sortedArr[i] = arr[i];
    }
    bubbleSort(sortedArr, n);

    printf("\n--- Performing Binary Search ---\n");
    printf("Sorted array for Binary Search: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", sortedArr[i]);
    }
    printf("\n");

    int binIdx = binarySearch(sortedArr, n, target);
    if (binIdx != -1) {
        printf("  [Binary Search] Element %d found at index %d (position %d) in sorted
array.\n", target, binIdx, binIdx + 1);
    } else {
        printf("  [Binary Search] Element %d not found in the array.\n", target);
    }

    return 0;
}
```

Figure 1.1: Terminal output for Linear and Binary Search execution (25BCP113)



Program 1.2: Bubble, Insertion and Selection Sort (E1Q2.c)

```c
#include <stdio.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
```

```
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        int temp = arr[minIdx];
        arr[minIdx] = arr[i];
        arr[i] = temp;
    }
}

int main() {
    printHeader();

    int n;
    printf("Enter array size: ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }

    int original[n], arrBubble[n], arrInsert[n], arrSelect[n];
    printf("Enter %d array elements:\n", n);
    for (int i = 0; i < n; i++) {
        printf("  Element [%d]: ", i + 1);
        scanf("%d", &original[i]);
        arrBubble[i] = original[i];
        arrInsert[i] = original[i];
        arrSelect[i] = original[i];
    }

    printf("\nOriginal Array: ");
    printArray(original, n);

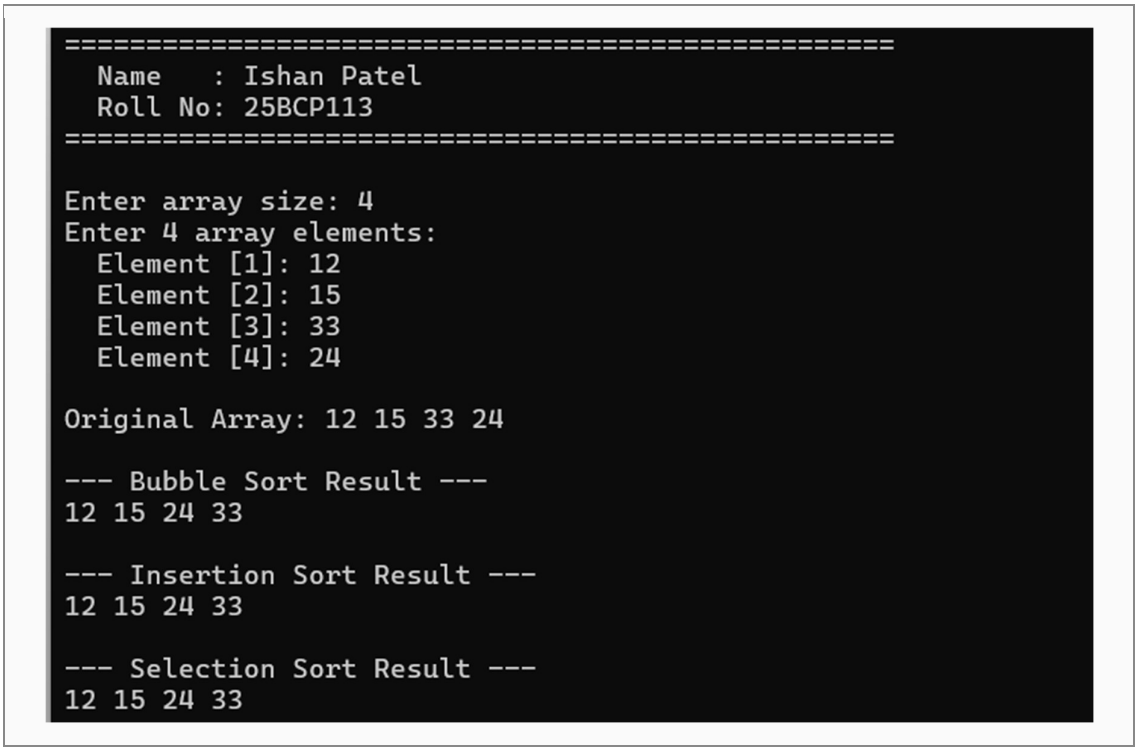
    // Bubble Sort
    bubbleSort(arrBubble, n);
    printf("\n--- Bubble Sort Result ---\n");
    printArray(arrBubble, n);

    // Insertion Sort
    insertionSort(arrInsert, n);
    printf("\n--- Insertion Sort Result ---\n");
    printArray(arrInsert, n);

    // Selection Sort
    selectionSort(arrSelect, n);
    printf("\n--- Selection Sort Result ---\n");
    printArray(arrSelect, n);

    return 0;
}
```

Figure 1.2: Terminal output for Bubble, Insertion, and Selection Sort execution (25BCP113)



Program 1.3: Extremum Elements (Min, Max, 2nd Min, 2nd Max) (E1Q3.c)

```c
#include <stdio.h>
#include <limits.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

int main() {
    printHeader();

    int n;
    printf("Enter number of elements (minimum 2): ");
    if (scanf("%d", &n) != 1 || n < 2) {
        printf("Error: At least 2 elements are required.\n");
        return 1;
    }

    int arr[n];
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        printf("  Element [%d]: ", i + 1);
        scanf("%d", &arr[i]);
    }

    int minVal = INT_MAX, secondMin = INT_MAX;
    int maxVal = INT_MIN, secondMax = INT_MIN;

    for (int i = 0; i < n; i++) {
        // Find minimum and second minimum
```

```
        if (arr[i] < minVal) {
            secondMin = minVal;
            minVal = arr[i];
        } else if (arr[i] < secondMin && arr[i] != minVal) {
            secondMin = arr[i];
        }

        // Find maximum and second maximum
        if (arr[i] > maxVal) {
            secondMax = maxVal;
            maxVal = arr[i];
        } else if (arr[i] > secondMax && arr[i] != maxVal) {
            secondMax = arr[i];
        }
    }

    printf("\n================ Summary ================\n");
    printf("  Array Elements         : ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    printf("  Minimum Element        : %d\n", minVal);
    printf("  Maximum Element        : %d\n", maxVal);

    if (secondMin == INT_MAX) {
        printf("  Second Smallest Element: (Not present - all elements equal)\n");
    } else {
        printf("  Second Smallest Element: %d\n", secondMin);
    }

    if (secondMax == INT_MIN) {
        printf("  Second Largest Element : (Not present - all elements equal)\n");
    } else {
        printf("  Second Largest Element : %d\n", secondMax);
    }
    printf("=========================================\n");

    return 0;
}
```

Figure 1.3: Terminal output for Minimum, Maximum, and Second Extremum elements (25BCP113)

```
================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
================================================

Enter number of elements (minimum 2): 4
Enter 4 elements:
  Element [1]: 25
  Element [2]: 63
  Element [3]: 54
  Element [4]: 21

================ Summary ================
  Array Elements         : 25 63 54 21
  Minimum Element        : 21
  Maximum Element        : 63
  Second Smallest Element: 25
  Second Largest Element : 54
=========================================
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• Linear Search runs in O(N) time without requiring sorted order, whereas Binary Search achieves O(log N) logarithmic time but strictly requires a pre-sorted array.

• Bubble, Insertion, and Selection Sort each have O(N^2) worst-case time complexity, with Insertion Sort running in O(N) linear time on nearly-sorted data.

• Single-pass extremum identification maintains min, max, second min, and second max in O(N) time with O(1) auxiliary space.

EXPERIMENT NO. 2

Revision of Structures (Student Records & Nested Organization-Employee)

1. Objective

To study and implement C structures for modeling composite user-defined datatypes, arrays of structures for handling multiple entity records, and nested structures to capture real-world hierarchical relationships.

2. Problem Statements / Program Definitions

1. Create a structure Student in C with student name, student roll number and student address as its data members. Create the variable of type student and print the values.

2. Modify the above program to implement arrays of structure. Create an array of 5 students and print their values.

3. Create a structure Organization with organization name and organization ID as its data members. Next, create another structure Employee that is nested in structure Organization with employee ID, employee salary and employee name as its data members. Write a program in such a way that there are two organizations and each of these contains two employees.

3. Implementation & Output Snapshots

Program 2.1: Single Student Structure Implementation (E2Q1.c)

```
#include <stdio.h>
#include <string.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct student {
    char name[50];
    int rollnum;
    char add[100];
};

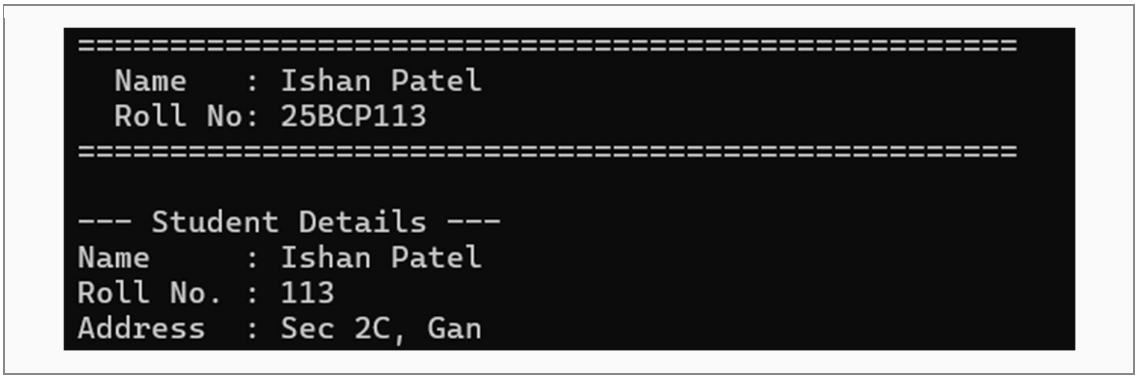
int main() {
    printHeader();

    struct student s1;
    strcpy(s1.name, "Ishan Patel");
    s1.rollnum = 113;
    strcpy(s1.add, "Sec 2C, Gan");

    printf("--- Student Details ---\n");
    printf("Name     : %s\n", s1.name);
    printf("Roll No. : %d\n", s1.rollnum);
    printf("Address  : %s\n", s1.add);

    return 0;
}
```

Figure 2.1: Terminal output for single Student structure instantiation (25BCP113)



Program 2.2: Array of 5 Student Structures (E2Q2.c)

```c
#include <stdio.h>
#include <string.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct student {
    char name[50];
    int rollnum;
    char add[100];
};

int main() {
    printHeader();

    int i = 0;
    struct student s[5];

    //------- Taking The data -------
    printf("--- Enter Details for 5 Students ---\n");
    for(i = 0; i < 5; i++) {
        printf("\nStudent %d:\n", i + 1);
        printf("  Enter Name    : ");
        scanf(" %[^\n]", s[i].name);

        printf("  Enter Roll No.: ");
        scanf("%d", &s[i].rollnum);

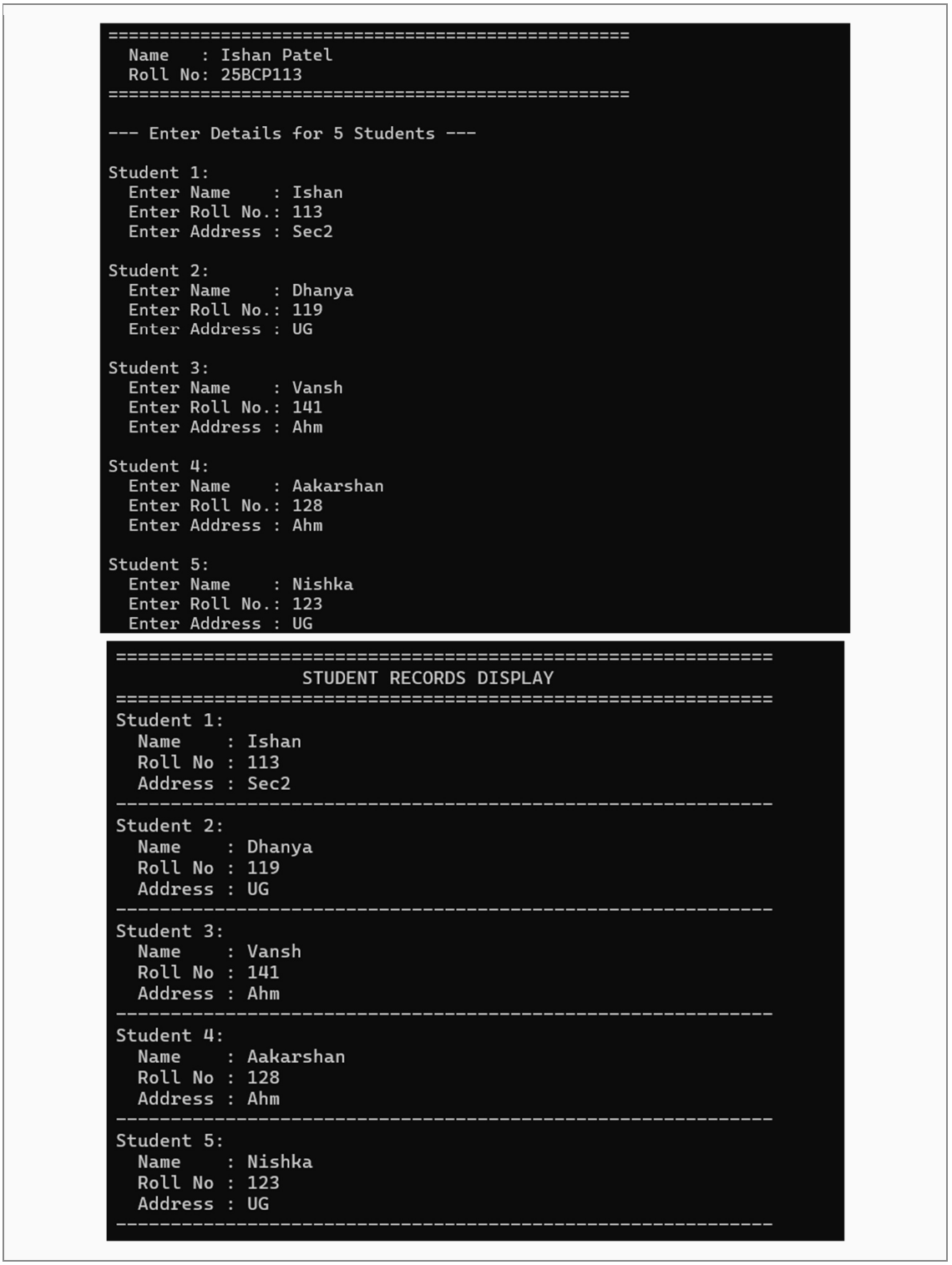
        printf("  Enter Address : ");
        scanf(" %[^\n]", s[i].add);
    }

    //------- Displaying The Data -------
    printf("\n============================================================\n");
    printf("                 STUDENT RECORDS DISPLAY                    \n");
    printf("============================================================\n");
    for(i = 0; i < 5; i++) {
        printf("Student %d:\n", i + 1);
        printf("  Name    : %s\n", s[i].name);
        printf("  Roll No : %d\n", s[i].rollnum);
        printf("  Address : %s\n", s[i].add);
        printf("------------------------------------------------------------\n");
```

```
    }

    return 0;
}
```

Figure 2.2: Terminal output for Array of 5 Student records display (25BCP113)



Program 2.3: Nested Structures (Organization & Employee) (E2Q3.c)

```
#include <stdio.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct Employee {
    char empID[20];
    float salary;
    char empName[50];
};

struct Organization {
    char orgName[50];
    int orgID;
    struct Employee emp[2];
};

int main() {
    printHeader();

    struct Organization org[2];
    int i, j;

    //-------- Input loop for Org -------------
    for(i = 0; i < 2; i++) {
        printf("============================================================\n");
        printf("  Enter Details for Organization %d\n", i + 1);
        printf("============================================================\n");
        printf("Enter Organization Name : ");
        scanf(" %[^\n]", org[i].orgName);
        printf("Enter Organization ID   : ");
        scanf("%d", &org[i].orgID);

        //-------- Input loop for emp -------------
        for(j = 0; j < 2; j++) {
            printf("\n  -- Enter Details for Employee %d of %s --\n", j + 1,
org[i].orgName);
            printf("  Enter Employee Name   : ");
            scanf(" %[^\n]", org[i].emp[j].empName);
            printf("  Enter Employee ID     : ");
            scanf(" %[^\n]", org[i].emp[j].empID);
            printf("  Enter Monthly Salary  : ");
            scanf("%f", &org[i].emp[j].salary);
        }
        printf("\n");
    }

    //-------- Output Loop ------------
    printf("\n============================================================\n");
    printf("              ORGANIZATION & EMPLOYEE RECORDS               \n");
    printf("============================================================\n");
    for(i = 0; i < 2; i++) {
        printf("\nOrganization %d: %s (ID: %d)\n", i + 1, org[i].orgName, org[i].orgID);
        printf("------------------------------------------------------------\n");

        for(j = 0; j < 2; j++) {
            printf("  Employee %d:\n", j + 1);
            printf("    Name   : %s\n", org[i].emp[j].empName);
```

```
            printf("    ID     : %s\n", org[i].emp[j].empID);
            printf("    Salary : Rs. %.2f\n", org[i].emp[j].salary);
        }
        printf("------------------------------------------------------------\n");
    }

    return 0;
}
```

Figure 2.3: Terminal output for Nested Organization and Employee records (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

============================================================
  Enter Details for Organization 1
============================================================
Enter Organization Name : HBC Healthcare
Enter Organization ID   : 1

  -- Enter Details for Employee 1 of HBC Healthcare --
  Enter Employee Name   : Ishan
  Enter Employee ID     : 1
  Enter Monthly Salary  : 20000

  -- Enter Details for Employee 2 of HBC Healthcare --
  Enter Employee Name   : Jeet
  Enter Employee ID     : 2
  Enter Monthly Salary  : 20000

============================================================
  Enter Details for Organization 2
============================================================
Enter Organization Name : Medibuzz
Enter Organization ID   : 2

  -- Enter Details for Employee 1 of Medibuzz --
  Enter Employee Name   : Yatharth
  Enter Employee ID     : 1
  Enter Monthly Salary  : 20000

  -- Enter Details for Employee 2 of Medibuzz --
  Enter Employee Name   : Shreya
  Enter Employee ID     : 1
  Enter Monthly Salary  : 20000
```

```
============================================================
              ORGANIZATION & EMPLOYEE RECORDS
============================================================

Organization 1: HBC Healthcare (ID: 1)
------------------------------------------------------------
  Employee 1:
    Name   : Ishan
    ID     : 1
    Salary : Rs. 20000.00
  Employee 2:
    Name   : Jeet
    ID     : 2
    Salary : Rs. 20000.00
------------------------------------------------------------

Organization 2: Medibuzz (ID: 2)
------------------------------------------------------------
  Employee 1:
    Name   : Yatharth
    ID     : 1
    Salary : Rs. 20000.00
  Employee 2:
    Name   : Shreya
    ID     : 1
    Salary : Rs. 20000.00
------------------------------------------------------------
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• Structures group heterogeneous datatypes into contiguous memory blocks, providing object-like record encapsulation in C.

• Arrays of structures enable indexed, tabular manipulation of multiple records.

• Nested structures permit hierarchical composition (an Organization entity containing an array of Employee entities).

EXPERIMENT NO. 3

Revision of Pointers (Arrays of Pointers, Pointers to Arrays, Structures & Swapping)

1. Objective

To master pointer arithmetic and memory addressing in C, exploring pointers to arrays versus arrays of pointers, accessing structure members through pointers via the arrow (->) operator, and implementing pass-by-reference variable swapping.

2. Problem Statements / Program Definitions

1. Write a program in C to implement arrays of pointers and pointers to arrays.

2. Write a program in C to implement pointers to structures.

3. Write a program in C to perform swapping of two numbers by passing addresses of the variables to the functions.

3. Implementation & Output Snapshots

Program 3.1: Pointer to Array vs Array of Pointers (E3Q1.c)

```
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("=================================================\n");
    printf("  Name   : Ishan Patel                           \n");
    printf("  Roll No: 25BCP113                              \n");
    printf("=================================================\n\n");
}

int main() {
    printHeader();

    // 1. Pointer to an Array
    printf("-------------------------------------------------------------\n");
    printf(" Part 1: Pointer to an Array (int (*p)[4])\n");
    printf("-------------------------------------------------------------\n");
    int a[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    int (*p)[4] = a; // Pointer to an array of 4 integers

    printf("2D Array elements accessed via Pointer to Array (*(*(p + i) + j)):\n");
    for (int i = 0; i < 3; i++) {
        printf("  Row %d: ", i);
        for (int j = 0; j < 4; j++) {
            printf("%3d ", *(*(p + i) + j));
        }
        printf("\n");
    }

    // 2. Array of Pointers
    printf("\n-------------------------------------------------------------\n");
    printf(" Part 2: Array of Pointers (int *arr[3])\n");
    printf("-------------------------------------------------------------\n");
    int *arr[3]; // Array of 3 integer pointers (each points to dynamically allocated row)

    for (int i = 0; i < 3; i++) {
```

```
        arr[i] = (int *)malloc(4 * sizeof(int));
        if (arr[i] == NULL) {
            printf("Memory allocation failed!\n");
            return 1;
        }
        for (int j = 0; j < 4; j++) {
            arr[i][j] = a[i][j]; // Copy values
        }
    }

    printf("2D Array elements accessed via Array of Pointers (arr[i][j]):\n");
    for (int i = 0; i < 3; i++) {
        printf("  arr[%d] (Address: %p) -> ", i, (void *)arr[i]);
        for (int j = 0; j < 4; j++) {
            printf("%3d ", arr[i][j]);
        }
        printf("\n");
    }

    // Free allocated memory
    for (int i = 0; i < 3; i++) {
        free(arr[i]);
    }
    printf("\nDynamically allocated row memory successfully freed.\n");

    return 0;
}
```

Figure 3.1: Terminal output for Pointer to Array and Array of Pointers (25BCP113)

```
=====================================================
  Name    : Ishan Patel
  Roll No: 25BCP113
=====================================================

------------------------------------------------------------
 Part 1: Pointer to an Array (int (*p)[4])
------------------------------------------------------------
2D Array elements accessed via Pointer to Array (*(*(p + i) + j)):
  Row 0:   1   2   3   4
  Row 1:   5   6   7   8
  Row 2:   9  10  11  12

------------------------------------------------------------
 Part 2: Array of Pointers (int *arr[3])
------------------------------------------------------------
2D Array elements accessed via Array of Pointers (arr[i][j]):
  arr[0] (Address: 000001405676E210) ->   1   2   3   4
  arr[1] (Address: 000001405676E2B0) ->   5   6   7   8
  arr[2] (Address: 000001405676E0B0) ->   9  10  11  12

Dynamically allocated row memory successfully freed.
```

Program 3.2: Pointer to Structure using Arrow (->) Operator (E3Q2.c)

```
#include <stdio.h>
#include <string.h>

void printHeader() {
```

```
    printf("================================================\n");
    printf("  Name   : Ishan Patel                          \n");
    printf("  Roll No: 25BCP113                            \n");
    printf("================================================\n\n");
}

struct Student {
    int id;
    char name[50];
    float score;
};

int main() {
    printHeader();

    struct Student s1;
    struct Student *ptr = &s1;

    printf("--- Enter Student Details ---\n");
    printf("Enter Student ID    : ");
    if (scanf("%d", &ptr->id) != 1) return 1;

    printf("Enter Student Name  : ");
    scanf(" %[^\n]", ptr->name);

    printf("Enter Student Score : ");
    if (scanf("%f", &ptr->score) != 1) return 1;

    printf("\n============================================================\n");
    printf("      STUDENT DETAILS ACCESSED VIA STRUCTURE POINTER        \n");
    printf("============================================================\n");
    printf("  Structure Address  : %p\n", (void *)ptr);
    printf("  Student ID (->id)  : %d\n", ptr->id);
    printf("  Student Name       : %s\n", ptr->name);
    printf("  Student Score      : %.2f\n", ptr->score);
    printf("============================================================\n");

    return 0;
}
```

Figure 3.2: Terminal output for Structure Pointer member access via -> operator (25BCP113)

```
================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
================================================

--- Enter Student Details ---
Enter Student ID    : 113
Enter Student Name  : Ishan
Enter Student Score : 63

============================================================
      STUDENT DETAILS ACCESSED VIA STRUCTURE POINTER
============================================================
  Structure Address  : 000000E3EC9FF8B0
  Student ID (->id)  : 113
  Student Name       : Ishan
  Student Score      : 63.00
============================================================
```

Program 3.3: Pass-by-Reference Variable Swapping (E3Q3.c)

```c
#include <stdio.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    printHeader();

    int num1, num2;
    printf("Enter two numbers (separated by space): ");
    if (scanf("%d %d", &num1, &num2) != 2) {
        printf("Invalid input.\n");
        return 1;
    }

    printf("\n--- Before Swapping ---\n");
    printf("  Variable num1: %d  (Stored at address: %p)\n", num1, (void *)&num1);
    printf("  Variable num2: %d  (Stored at address: %p)\n", num2, (void *)&num2);

    // Call swap by passing addresses of variables
    swap(&num1, &num2);

    printf("\n--- After Swapping (via swap(&num1, &num2)) ---\n");
    printf("  Variable num1: %d\n", num1);
    printf("  Variable num2: %d\n", num2);
    printf("------------------------------------------------------------\n");

    return 0;
}
```

Figure 3.3: Terminal output for Address-based Variable Swapping (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

Enter two numbers (separated by space): 23 32

--- Before Swapping ---
  Variable num1: 23  (Stored at address: 000000178A1FFDCC)
  Variable num2: 32  (Stored at address: 000000178A1FFDC8)

--- After Swapping (via swap(&num1, &num2)) ---
  Variable num1: 32
  Variable num2: 23
------------------------------------------------------------
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• A pointer to an array (int (*p)[4]) references a fixed-column contiguous 2D block, whereas an array of pointers (int *arr[3]) allows dynamic allocation of separate rows (ragged arrays).

• Structure pointers avoid expensive structure copying across function calls, accessing fields directly in O(1) via ptr->member.

• Passing addresses enables true in-place swapping without global variables.

EXPERIMENT NO. 4

Linked List (Merge Sorted Lists, 3-Way Insertion/Deletion, Circular & Doubly LL)

1. Objective

To implement dynamic linear node-based data structures: merging two pre-sorted singly linked lists in O(N+M) time, 3-way node insertion (front, after node, end), 3-way node deletion (beginning, middle, end), Circular Linked List traversal, and Doubly Linked List bidirectional traversal.

2. Problem Statements / Program Definitions

1. Write a program that takes two sorted lists as inputs and merge them into one sorted list. (e.g. A: 5 => 10 => 15, B: 2 => 3 => 20 -> Output: 2 => 3 => 5 => 10 => 15 => 20).

2. Write a program to insert a new node into the linked list using three ways: (a) At front, (b) After a given node, (c) At the end.

3. Write a program to delete a node from the linked list using three ways: (a) Delete from beginning, (b) Delete from end, (c) Delete from middle.

4. Implement the circular linked list and perform the operation of traversal on it.

5. Implement the doubly linked list and perform deletion and/or insertion operations on it.

3. Implementation & Output Snapshots

Program 4.1: Merge Two Sorted Linked Lists (E4Q1.c)

```
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("=================================================\n");
    printf("  Name   : Ishan Patel                           \n");
    printf("  Roll No: 25BCP113                              \n");
    printf("=================================================\n\n");
}

struct Node {
    int val;
    struct Node* next;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->val = value;
    newNode->next = NULL;
    return newNode;
}

void printList(struct Node* head) {
    if (head == NULL) {
        printf("Empty List\n");
        return;
    }
    struct Node* curr = head;
    while (curr != NULL) {
        printf("%d", curr->val);
        if (curr->next != NULL) printf(" => ");
```

```
        curr = curr->next;
    }
    printf("\n");
}

void freeList(struct Node* head) {
    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    printHeader();

    int sizeA, sizeB, value;
    struct Node *list_A = NULL, *tailA = NULL;
    struct Node *list_B = NULL, *tailB = NULL;

    printf("--- Input for Sorted List A ---\n");
    printf("Enter number of elements for List A: ");
    if (scanf("%d", &sizeA) != 1 || sizeA < 0) return 1;

    for (int i = 0; i < sizeA; i++) {
        printf("  Enter element %d (in ascending order): ", i + 1);
        scanf("%d", &value);
        struct Node* newNode = createNode(value);
        if (list_A == NULL) {
            list_A = newNode;
            tailA = newNode;
        } else {
            tailA->next = newNode;
            tailA = newNode;
        }
    }

    printf("\n--- Input for Sorted List B ---\n");
    printf("Enter number of elements for List B: ");
    if (scanf("%d", &sizeB) != 1 || sizeB < 0) return 1;

    for (int i = 0; i < sizeB; i++) {
        printf("  Enter element %d (in ascending order): ", i + 1);
        scanf("%d", &value);
        struct Node* newNode = createNode(value);
        if (list_B == NULL) {
            list_B = newNode;
            tailB = newNode;
        } else {
            tailB->next = newNode;
            tailB = newNode;
        }
    }

    printf("\n-------------------------------------------------------------\n");
    printf("List A: ");
    printList(list_A);

    printf("List B: ");
    printList(list_B);
    printf("-------------------------------------------------------------\n");

    // Merge logic using dummy node
    struct Node dummy;
    struct Node* tail = &dummy;
```

```
    dummy.next = NULL;

    struct Node* p1 = list_A;
    struct Node* p2 = list_B;

    while (p1 != NULL && p2 != NULL) {
        if (p1->val <= p2->val) {
            tail->next = p1;
            p1 = p1->next;
        } else {
            tail->next = p2;
            p2 = p2->next;
        }
        tail = tail->next;
    }

    if (p1 != NULL) {
        tail->next = p1;
    } else {
        tail->next = p2;
    }

    struct Node* merged_list = dummy.next;

    printf("\nMerged Sorted List: ");
    printList(merged_list);
    printf("------------------------------------------------------------\n");

    freeList(merged_list);
    return 0;
}
```

Figure 4.1: Terminal output for Merging two sorted Linked Lists (25BCP113)

```
==================================================
  Name    : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Input for Sorted List A ---
Enter number of elements for List A: 2
  Enter element 1 (in ascending order): 21
  Enter element 2 (in ascending order): 35

--- Input for Sorted List B ---
Enter number of elements for List B: 3
  Enter element 1 (in ascending order): 11
  Enter element 2 (in ascending order): 26
  Enter element 3 (in ascending order): 47

------------------------------------------------------------
List A: 21 => 35
List B: 11 => 26 => 47
------------------------------------------------------------

Merged Sorted List: 11 => 21 => 26 => 35 => 47
------------------------------------------------------------
```

Program 4.2: 3-Way Node Insertion in Singly Linked List (E4Q2.c)

```c
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("================================================\n");
    printf("  Name   : Ishan Patel                          \n");
    printf("  Roll No: 25BCP113                             \n");
    printf("================================================\n\n");
}

struct Node {
    int val;
    struct Node* next;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->val = value;
    newNode->next = NULL;
    return newNode;
}

// 1. Insert at the front of the list
void insertAtFront(struct Node** head_ref, int value) {
    struct Node* newNode = createNode(value);
    newNode->next = *head_ref;
    *head_ref = newNode;
    printf(">> Inserted [%d] at the front.\n", value);
}

// 2. Insert after a given node with target value
void insertAfter(struct Node* head, int targetVal, int newValue) {
    struct Node* curr = head;
    while (curr != NULL && curr->val != targetVal) {
        curr = curr->next;
    }

    if (curr == NULL) {
        printf(">> Target node with value %d not found in the list!\n", targetVal);
        return;
    }

    struct Node* newNode = createNode(newValue);
    newNode->next = curr->next;
    curr->next = newNode;
    printf(">> Inserted [%d] after node [%d].\n", newValue, targetVal);
}

// 3. Insert at the end of the list
void insertAtEnd(struct Node** head_ref, int value) {
    struct Node* newNode = createNode(value);
    if (*head_ref == NULL) {
        *head_ref = newNode;
        printf(">> Inserted [%d] at the end (first node).\n", value);
        return;
    }

    struct Node* curr = *head_ref;
```

```
    while (curr->next != NULL) {
        curr = curr->next;
    }
    curr->next = newNode;
    printf(">> Inserted [%d] at the end.\n", value);
}

void printList(struct Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }
    struct Node* curr = head;
    while (curr != NULL) {
        printf("[%d]", curr->val);
        if (curr->next != NULL) printf(" -> ");
        curr = curr->next;
    }
    printf(" -> NULL\n");
}

void freeList(struct Node* head) {
    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    printHeader();

    struct Node* head = NULL;

    printf("Demonstrating all 3 insertion methods:\n");
    printf("-------------------------------------------------------------\n");

    // a. At front
    printf("(a) Insertion at Front:\n");
    insertAtFront(&head, 15);
    insertAtFront(&head, 10);
    insertAtFront(&head, 5);
    printf("Current List: ");
    printList(head);

    // b. After a given node
    printf("\n(b) Insertion After a Given Node:\n");
    insertAfter(head, 10, 12); // Insert 12 after 10
    printf("Current List: ");
    printList(head);

    // c. At end
    printf("\n(c) Insertion at End:\n");
    insertAtEnd(&head, 20);
    insertAtEnd(&head, 25);
    printf("Current List: ");
    printList(head);

    printf("-------------------------------------------------------------\n");
    printf("Final Linked List: ");
    printList(head);
    printf("=============================================================\n");

    freeList(head);
    return 0;
```

```
}
```

Figure 4.2: Terminal output for 3-Way Linked List Insertion (25BCP113)

```
==========================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==========================================================

Demonstrating all 3 insertion methods:
------------------------------------------------------------
(a) Insertion at Front:
>> Inserted [15] at the front.
>> Inserted [10] at the front.
>> Inserted [5] at the front.
Current List: [5] -> [10] -> [15] -> NULL

(b) Insertion After a Given Node:
>> Inserted [12] after node [10].
Current List: [5] -> [10] -> [12] -> [15] -> NULL

(c) Insertion at End:
>> Inserted [20] at the end.
>> Inserted [25] at the end.
Current List: [5] -> [10] -> [12] -> [15] -> [20] -> [25] -> NULL
------------------------------------------------------------
Final Linked List: [5] -> [10] -> [12] -> [15] -> [20] -> [25] -> NULL
============================================================
```

Program 4.3: 3-Way Node Deletion in Singly Linked List (E4Q3.c)

```c
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("==========================================================\n");
    printf("  Name   : Ishan Patel                                    \n");
    printf("  Roll No: 25BCP113                                       \n");
    printf("==========================================================\n\n");
}

struct Node {
    int data;
    struct Node *next;
};

struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

void printList(struct Node* head) {
    if (head == NULL) {
        printf("Empty List (NULL)\n");
```

```
        return;
    }
    struct Node* temp = head;
    while (temp != NULL) {
        printf("[%d]", temp->data);
        if (temp->next != NULL) printf(" -> ");
        temp = temp->next;
    }
    printf(" -> NULL\n");
}

// a. Delete from the beginning
void deleteFromBeginning(struct Node** head_ref) {
    if (*head_ref == NULL) {
        printf(">> Underflow: List is empty, cannot delete.\n");
        return;
    }
    struct Node* temp = *head_ref;
    *head_ref = (*head_ref)->next;
    printf(">> Deleted [%d] from beginning.\n", temp->data);
    free(temp);
}

// b. Delete from the end
void deleteFromEnd(struct Node** head_ref) {
    if (*head_ref == NULL) {
        printf(">> Underflow: List is empty, cannot delete.\n");
        return;
    }
    if ((*head_ref)->next == NULL) {
        printf(">> Deleted [%d] from end (was only node).\n", (*head_ref)->data);
        free(*head_ref);
        *head_ref = NULL;
        return;
    }
    struct Node* temp = *head_ref;
    struct Node* prev = NULL;
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = NULL;
    printf(">> Deleted [%d] from end.\n", temp->data);
    free(temp);
}

// c. Delete from the middle (by 1-based position)
void deleteFromMiddle(struct Node** head_ref, int position) {
    if (*head_ref == NULL) {
        printf(">> Underflow: List is empty.\n");
        return;
    }
    if (position == 1) {
        deleteFromBeginning(head_ref);
        return;
    }

    struct Node* temp = *head_ref;
    struct Node* prev = NULL;

    for (int i = 1; i < position && temp != NULL; i++) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == NULL) {
        printf(">> Position %d is out of range.\n", position);
```

```
        return;
    }

    prev->next = temp->next;
    printf(">> Deleted [%d] from position %d.\n", temp->data, position);
    free(temp);
}

void freeList(struct Node* head) {
    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    printHeader();

    struct Node* head = NULL;
    int n, val;

    printf("Enter number of nodes to create initial list (e.g., 5): ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    struct Node* tail = NULL;
    for (int i = 0; i < n; i++) {
        printf("  Enter data for node %d: ", i + 1);
        scanf("%d", &val);
        struct Node* newNode = createNode(val);
        if (head == NULL) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
    }

    printf("\nInitial Linked List: ");
    printList(head);
    printf("------------------------------------------------------------\n");

    // a. Delete from beginning
    printf("\n(a) Delete from Beginning:\n");
    deleteFromBeginning(&head);
    printf("List after deletion: ");
    printList(head);

    // b. Delete from end
    printf("\n(b) Delete from End:\n");
    deleteFromEnd(&head);
    printf("List after deletion: ");
    printList(head);

    // c. Delete from middle
    int pos;
    printf("\n(c) Delete from Middle:\n");
    printf("Enter position to delete: ");
    if (scanf("%d", &pos) == 1) {
        deleteFromMiddle(&head, pos);
        printf("List after deletion: ");
        printList(head);
    }
```

```
    printf("============================================================\n");
    freeList(head);
    return 0;
}
```

Figure 4.3: Terminal output for 3-Way Linked List Deletion (25BCP113)

```
==================================================
  Name    : Ishan Patel
  Roll No: 25BCP113
==================================================

Enter number of nodes to create initial list (e.g., 5): 4
  Enter data for node 1: 13
  Enter data for node 2: 26
  Enter data for node 3: 45
  Enter data for node 4: 68

Initial Linked List: [13] -> [26] -> [45] -> [68] -> NULL
------------------------------------------------------------

(a) Delete from Beginning:
>> Deleted [13] from beginning.
List after deletion: [26] -> [45] -> [68] -> NULL

(b) Delete from End:
>> Deleted [68] from end.
List after deletion: [26] -> [45] -> NULL

(c) Delete from Middle:
Enter position to delete: 2
>> Deleted [45] from position 2.
List after deletion: [26] -> NULL
==============================================================
```

Program 4.4: Circular Linked List Implementation & Traversal (E4Q4.c)

```
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct Node {
    int data;
    struct Node* next;
};

void append(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    if (new_node == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
```

```
    new_node->data = new_data;
    new_node->next = NULL;

    if (*head_ref == NULL) {
        *head_ref = new_node;
        new_node->next = *head_ref; // Points to itself
        return;
    }

    struct Node* last = *head_ref;
    while (last->next != *head_ref) {
        last = last->next;
    }

    last->next = new_node;
    new_node->next = *head_ref; // Maintains circular structure
}

void traverse(struct Node* head) {
    if (head == NULL) {
        printf("The Circular Linked List is empty.\n");
        return;
    }

    struct Node* current = head;
    printf("Start (Head) -> ");

    do {
        printf("[%d] -> ", current->data);
        current = current->next;
    } while (current != head);

    printf("(Back to Head: [%d])\n", head->data);
}

void freeCircularList(struct Node** head_ref) {
    if (*head_ref == NULL) return;

    struct Node* current = *head_ref;
    struct Node* next_node;
    do {
        next_node = current->next;
        free(current);
        current = next_node;
    } while (current != *head_ref);

    *head_ref = NULL;
}

int main() {
    printHeader();

    struct Node* head = NULL;

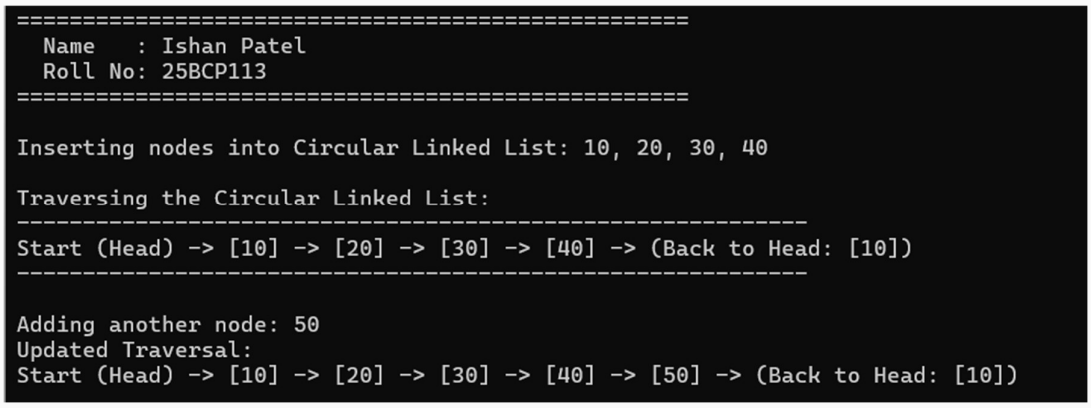
    printf("Inserting nodes into Circular Linked List: 10, 20, 30, 40\n");
    append(&head, 10);
    append(&head, 20);
    append(&head, 30);
    append(&head, 40);

    printf("\nTraversing the Circular Linked List:\n");
    printf("-------------------------------------------------------------\n");
    traverse(head);
    printf("-------------------------------------------------------------\n");
```

```
    printf("\nAdding another node: 50\n");
    append(&head, 50);
    printf("Updated Traversal:\n");
    traverse(head);

    freeCircularList(&head);
    return 0;
}
```

Figure 4.4: Terminal output for Circular Linked List Traversal (25BCP113)



Program 4.5: Doubly Linked List Bidirectional Operations (E4Q5.c)

```
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

// 1. Insert at front
void insertAtFront(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = *head_ref;
    new_node->prev = NULL;

    if (*head_ref != NULL) {
        (*head_ref)->prev = new_node;
    }

    *head_ref = new_node;
    printf(">> Inserted [%d] at Front.\n", new_data);
}

// 2. Insert after a given node
```

```
void insertAfter(struct Node* prev_node, int new_data) {
    if (prev_node == NULL) {
        printf(">> Error: The given previous node cannot be NULL.\n");
        return;
    }

    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = prev_node->next;
    new_node->prev = prev_node;

    if (prev_node->next != NULL) {
        prev_node->next->prev = new_node;
    }
    prev_node->next = new_node;

    printf(">> Inserted [%d] after [%d].\n", new_data, prev_node->data);
}

// 3. Insert at end
void insertAtEnd(struct Node** head_ref, int new_data) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    new_node->data = new_data;
    new_node->next = NULL;

    if (*head_ref == NULL) {
        new_node->prev = NULL;
        *head_ref = new_node;
        printf(">> Inserted [%d] at End (First Node).\n", new_data);
        return;
    }

    struct Node* last = *head_ref;
    while (last->next != NULL) {
        last = last->next;
    }

    last->next = new_node;
    new_node->prev = last;
    printf(">> Inserted [%d] at End.\n", new_data);
}

// Delete a given node
void deleteNode(struct Node** head_ref, struct Node* del_node) {
    if (*head_ref == NULL || del_node == NULL) {
        return;
    }

    if (*head_ref == del_node) {
        *head_ref = del_node->next;
    }

    if (del_node->next != NULL) {
        del_node->next->prev = del_node->prev;
    }

    if (del_node->prev != NULL) {
        del_node->prev->next = del_node->next;
    }

    printf(">> Deleted node [%d].\n", del_node->data);
    free(del_node);
}

void printList(struct Node* head) {
```

```
    struct Node* last = NULL;
    struct Node* curr = head;

    printf("  Forward  : NULL <-> ");
    while (curr != NULL) {
        printf("[%d] <-> ", curr->data);
        last = curr;
        curr = curr->next;
    }
    printf("NULL\n");

    printf("  Backward : NULL <-> ");
    while (last != NULL) {
        printf("[%d] <-> ", last->data);
        last = last->prev;
    }
    printf("NULL\n");
}

void freeList(struct Node* head) {
    struct Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next;
        free(temp);
    }
}

int main() {
    printHeader();

    struct Node* head = NULL;

    printf("--- 1. Doubly Linked List Insertions ---\n");
    insertAtFront(&head, 20);
    insertAtFront(&head, 10);
    insertAtEnd(&head, 40);
    insertAfter(head->next, 30); // Insert 30 after 20
    insertAtEnd(&head, 50);

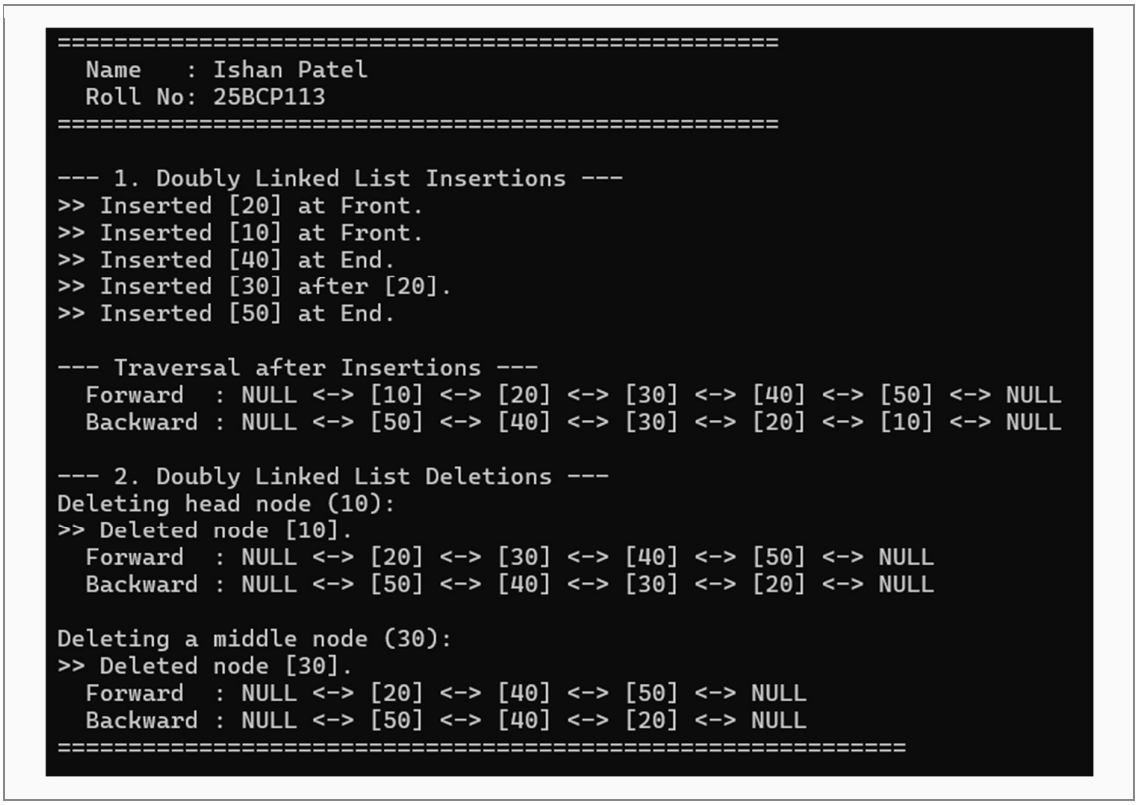
    printf("\n--- Traversal after Insertions ---\n");
    printList(head);

    printf("\n--- 2. Doubly Linked List Deletions ---\n");
    printf("Deleting head node (10):\n");
    deleteNode(&head, head);
    printList(head);

    printf("\nDeleting a middle node (30):\n");
    deleteNode(&head, head->next); // points to 30
    printList(head);

    printf("============================================================\n");
    freeList(head);
    return 0;
}
```

Figure 4.5: Terminal output for Doubly Linked List Bidirectional Operations (25BCP113)



4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• Singly Linked Lists provide dynamic sizing with O(1) insertion/deletion at head, avoiding contiguous reallocation penalties.

• Merging sorted lists re-links existing nodes in O(N+M) time using O(1) auxiliary space.

• Circular Linked Lists connect tail to head, ideal for round-robin scheduling without encountering NULL pointers.

• Doubly Linked Lists allow O(1) bidirectional node deletion given a pointer to that node.

EXPERIMENT NO. 5

Stack (Array & Linked List Implementation, Recursive Stack Sorting)

1. Objective

To implement the Last-In-First-Out (LIFO) Stack abstract data type using bounded contiguous arrays and unbounded dynamic linked lists, and to solve recursive stack sorting strictly without loop constructs.

2. Problem Statements / Program Definitions

1. Implement a stack using an array and using a linked list.

2. Given a stack, sort it using recursion. Use of any loop constructs like while, for, etc. is not allowed. We can only use the following functions on Stack S: (a) isEmpty(S), (b) push(S), (c) pop(S), (d) top(S).

3. Implementation & Output Snapshots

Program 5.1: Stack Implementation using Array and Linked List (E5Q1.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("=====================================================\n");
    printf("  Name   : Ishan Patel                               \n");
    printf("  Roll No: 25BCP113                                  \n");
    printf("=====================================================\n\n");
}

/* ============================================================
   PART 1: STACK IMPLEMENTATION USING ARRAY
   ============================================================ */
#define MAX_SIZE 5

struct ArrayStack {
    int items[MAX_SIZE];
    int top;
};

void initArrayStack(struct ArrayStack* s) {
    s->top = -1;
}

bool isArrayStackFull(struct ArrayStack* s) {
    return s->top == MAX_SIZE - 1;
}

bool isArrayStackEmpty(struct ArrayStack* s) {
    return s->top == -1;
}

void pushArray(struct ArrayStack* s, int val) {
    if (isArrayStackFull(s)) {
        printf("  [Array Stack] Overflow! Cannot push %d.\n", val);
        return;
    }
    s->items[++(s->top)] = val;
    printf("  [Array Stack] Pushed %d (Top index: %d)\n", val, s->top);
}

int popArray(struct ArrayStack* s) {
    if (isArrayStackEmpty(s)) {
```

```
        printf("  [Array Stack] Underflow! Stack is empty.\n");
        return -1;
    }
    return s->items[(s->top)--];
}

int peekArray(struct ArrayStack* s) {
    if (isArrayStackEmpty(s)) {
        printf("  [Array Stack] Stack is empty.\n");
        return -1;
    }
    return s->items[s->top];
}

void displayArrayStack(struct ArrayStack* s) {
    if (isArrayStackEmpty(s)) {
        printf("  [Array Stack] Stack is empty.\n");
        return;
    }
    printf("  [Array Stack] Top -> ");
    for (int i = s->top; i >= 0; i--) {
        printf("[%d] ", s->items[i]);
    }
    printf("\n");
}

/* ============================================================
   PART 2: STACK IMPLEMENTATION USING LINKED LIST
   ============================================================ */
struct Node {
    int data;
    struct Node* next;
};

bool isListEmpty(struct Node* top) {
    return top == NULL;
}

void pushList(struct Node** top_ref, int value) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    if (new_node == NULL) {
        printf("  [List Stack] Heap Overflow! Cannot allocate memory.\n");
        return;
    }
    new_node->data = value;
    new_node->next = *top_ref;
    *top_ref = new_node;
    printf("  [List Stack] Pushed %d onto Linked List Stack\n", value);
}

int popList(struct Node** top_ref) {
    if (isListEmpty(*top_ref)) {
        printf("  [List Stack] Underflow! Stack is empty.\n");
        return -1;
    }
    struct Node* temp = *top_ref;
    int popped_value = temp->data;
    *top_ref = (*top_ref)->next;
    free(temp);
    return popped_value;
}

int peekList(struct Node* top) {
    if (isListEmpty(top)) {
        printf("  [List Stack] Stack is empty.\n");
        return -1;
    }
```

```
    return top->data;
}

void displayListStack(struct Node* top) {
    if (isListEmpty(top)) {
        printf("  [List Stack] Stack is empty.\n");
        return;
    }
    printf("  [List Stack] Top -> ");
    struct Node* curr = top;
    while (curr != NULL) {
        printf("[%d] -> ", curr->data);
        curr = curr->next;
    }
    printf("NULL\n");
}

int main() {
    printHeader();

    // 1. Array Stack Testing
    printf("============================================================\n");
    printf("           1. TESTING STACK USING ARRAY                     \n");
    printf("============================================================\n");
    struct ArrayStack aStack;
    initArrayStack(&aStack);

    pushArray(&aStack, 10);
    pushArray(&aStack, 20);
    pushArray(&aStack, 30);
    pushArray(&aStack, 40);
    pushArray(&aStack, 50);
    pushArray(&aStack, 60); // Overflow test
    displayArrayStack(&aStack);

    printf("  Top Element (Peek): %d\n", peekArray(&aStack));
    printf("  Popped: %d\n", popArray(&aStack));
    printf("  Popped: %d\n", popArray(&aStack));
    displayArrayStack(&aStack);

    // 2. Linked List Stack Testing
    printf("\n============================================================\n");
    printf("        2. TESTING STACK USING LINKED LIST                  \n");
    printf("============================================================\n");
    struct Node* lStack = NULL;

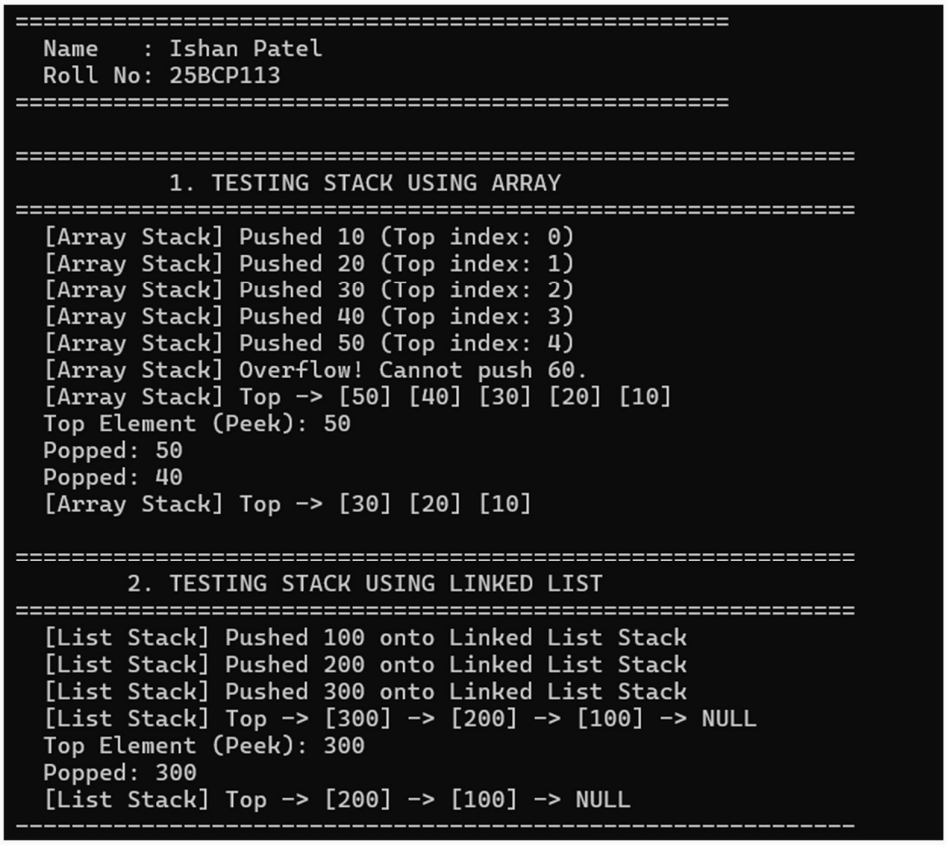
    pushList(&lStack, 100);
    pushList(&lStack, 200);
    pushList(&lStack, 300);
    displayListStack(lStack);

    printf("  Top Element (Peek): %d\n", peekList(lStack));
    printf("  Popped: %d\n", popList(&lStack));
    displayListStack(lStack);

    while (!isListEmpty(lStack)) {
        popList(&lStack);
    }
    printf("------------------------------------------------------------\n");

    return 0;
}
```

Figure 5.1: Terminal output for Array and Linked List Stack Operations (25BCP113)



Program 5.2: Recursive Stack Sorting Without Loop Constructs (E5Q2.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct Node {
    int data;
    struct Node* next;
};

// a. isEmpty(S): Tests whether stack is empty or not
bool isEmpty(struct Node* top) {
    return top == NULL;
}

// b. push(S): Adds new element to the stack
void push(struct Node** top_ref, int value) {
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));
    if (new_node == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
```

```
    }
    new_node->data = value;
    new_node->next = *top_ref;
    *top_ref = new_node;
}

// c. pop(S): Removes top element from the stack
int pop(struct Node** top_ref) {
    if (isEmpty(*top_ref)) {
        printf("Stack Underflow!\n");
        return -1;
    }
    struct Node* temp = *top_ref;
    int popped_value = temp->data;
    *top_ref = (*top_ref)->next;
    free(temp);
    return popped_value;
}

// d. top(S): Returns value of the top element
int top(struct Node* top) {
    if (isEmpty(top)) {
        printf("Stack is empty!\n");
        return -1;
    }
    return top->data;
}

// Recursive helper: insert element into sorted stack without any loops
void sortedInsert(struct Node** top_ref, int element) {
    // Base case: Stack is empty or element is greater than top element
    if (isEmpty(*top_ref) || element > top(*top_ref)) {
        push(top_ref, element);
        return;
    }

    // If top is greater, remove top and recurse
    int temp = pop(top_ref);
    sortedInsert(top_ref, element);

    // Put back the top item
    push(top_ref, temp);
}

// Recursive stack sorting function (strictly no loops)
void sortStack(struct Node** top_ref) {
    // Base case: Stack is empty
    if (isEmpty(*top_ref)) {
        return;
    }

    // Remove the top item
    int temp = pop(top_ref);

    // Sort the remaining stack recursively
    sortStack(top_ref);

    // Push the top item back in sorted order
    sortedInsert(top_ref, temp);
}

// Recursive print function without using any loops
void printStackRecursive(struct Node* curr) {
    if (isEmpty(curr)) {
        printf("NULL\n");
        return;
```

```
    }
    printf("[%d] -> ", curr->data);
    printStackRecursive(curr->next);
}

// Recursive cleanup
void freeStackRecursive(struct Node** top_ref) {
    if (isEmpty(*top_ref)) return;
    pop(top_ref);
    freeStackRecursive(top_ref);
}

int main() {
    printHeader();

    struct Node* S = NULL;

    printf("Pushing elements to stack: 30, -5, 18, 14, 45\n");
    push(&S, 30);
    push(&S, -5);
    push(&S, 18);
    push(&S, 14);
    push(&S, 45);

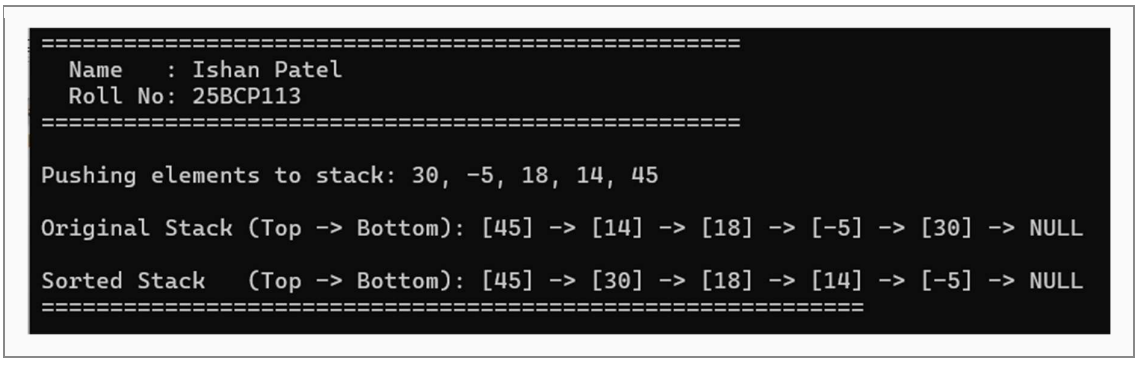
    printf("\nOriginal Stack (Top -> Bottom): ");
    printStackRecursive(S);

    // Sort the stack recursively without loops
    sortStack(&S);

    printf("\nSorted Stack   (Top -> Bottom): ");
    printStackRecursive(S);
    printf("============================================================\n");

    freeStackRecursive(&S);
    return 0;
}
```

Figure 5.2: Terminal output for Recursive Stack Sorting Execution (25BCP113)



4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

- Stacks enforce strict LIFO semantics; all operations (push, pop, peek) operate in O(1) time.
- Array stacks offer cache efficiency but have fixed capacity; linked-list stacks grow dynamically.
- Recursive stack sorting uses the runtime call stack as auxiliary storage to hold popped items and recursively insert each item in sorted order in O(N^2) time without loops.

EXPERIMENT NO. 6

Stack Applications (Infix to Postfix, Postfix Evaluation & Parentheses Validation)

1. Objective

To apply stacks to arithmetic expression parsing: transforming human-readable infix expressions to postfix notation via operator precedence and associativity, evaluating postfix arithmetic expressions, and validating balanced bracket syntax.

2. Problem Statements / Program Definitions

1. Convert the given infix expression into postfix expression using stack. Example: Input: a + b * (c ^ d - e) ^ (f + g * h) - i -> Output: a b c d ^ e - f g h * + ^ * + i -

2. Write a program to evaluate the following given postfix expressions: (a) 2 3 1 * + 9 - [Output: -4], (b) 2 2 + 2 / 5 * 7 + [Output: 17]

3. Given an expression, write a program to examine whether the pairs and the orders of '{', '}', '(', ')', '[', ']' are correct in the expression or not. Example: '[( )]{ }{[( )( )]( )}' -> Balanced; '[( ])' -> Not Balanced.

3. Implementation & Output Snapshots

Program 6.1: Infix to Postfix Expression Conversion (E6Q1.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

#define MAX 200

// Stack for characters
char stack[MAX];
int top = -1;

void push(char c) {
    if (top < MAX - 1) {
        stack[++top] = c;
    }
}

char pop() {
    if (top >= 0) {
        return stack[top--];
    }
    return '\0';
}

char peek() {
    if (top >= 0) {
        return stack[top];
    }
    return '\0';
}
```

```
int isEmpty() {
    return top == -1;
}

// Function to return precedence of operators
int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return 0;
    }
}

// Check associativity: '^' is right-associative (returns 1 for right, 0 for left)
int isRightAssociative(char op) {
    if (op == '^') return 1;
    return 0;
}

// Check if character is alphanumeric operand
int isOperand(char c) {
    return isalnum(c);
}

// Convert Infix to Postfix
void infixToPostfix(const char* infix, char* postfix) {
    top = -1;
    int k = 0;

    for (int i = 0; infix[i] != '\0'; i++) {
        char c = infix[i];

        if (isspace(c)) {
            continue; // Skip whitespace
        }

        if (isOperand(c)) {
            postfix[k++] = c;
            postfix[k++] = ' '; // add space for clean readability
        }
        else if (c == '(') {
            push(c);
        }
        else if (c == ')') {
            while (!isEmpty() && peek() != '(') {
                postfix[k++] = pop();
                postfix[k++] = ' ';
            }
            if (!isEmpty() && peek() == '(') {
                pop(); // discard '('
            }
        }
        else { // Operator encountered
            while (!isEmpty() && peek() != '(' &&
                  ((precedence(peek()) > precedence(c)) ||
                   (precedence(peek()) == precedence(c) && !isRightAssociative(c)))) {
                postfix[k++] = pop();
                postfix[k++] = ' ';
            }
            push(c);
```

```
        }
    }

    // Pop remaining operators in stack
    while (!isEmpty()) {
        char op = pop();
        if (op != '(') {
            postfix[k++] = op;
            postfix[k++] = ' ';
        }
    }

    if (k > 0 && postfix[k - 1] == ' ') {
        k--; // remove trailing space
    }
    postfix[k] = '\0';
}

int main() {
    printHeader();

    char postfix[MAX];

    // 1. Example from manual: a + b * (c ^ d - e) ^ (f + g * h) - i
    const char* manualExp = "a + b * (c ^ d - e) ^ (f + g * h) - i";
    printf("--- Test Case 1 (From Lab Manual) ---\n");
    printf("Infix Expression   : %s\n", manualExp);
    infixToPostfix(manualExp, postfix);
    printf("Postfix Expression : %s\n", postfix);

    // 2. User Input option
    printf("\n--- Test Case 2 (Interactive Input) ---\n");
    char userInfix[MAX];
    printf("Enter an infix expression (e.g. (A+B)*(C-D)): ");
    if (fgets(userInfix, sizeof(userInfix), stdin) != NULL) {
        userInfix[strcspn(userInfix, "\r\n")] = '\0';
        if (strlen(userInfix) > 0) {
            infixToPostfix(userInfix, postfix);
            printf("Postfix Expression : %s\n", postfix);
        }
    }
    printf("============================================================\n");

    return 0;
}
```

Figure 6.1: Terminal output for Infix to Postfix Expression Conversion (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Test Case 1 (From Lab Manual) ---
Infix Expression   : a + b * (c ^ d - e) ^ (f + g * h) - i
Postfix Expression : a b c d ^ e - f g h * + ^ * + i -

--- Test Case 2 (Interactive Input) ---
Enter an infix expression (e.g. (A+B)*(C-D)): (A+B)*(C-D)
Postfix Expression : A B + C D - *
============================================================
```

Program 6.2: Postfix Expression Arithmetic Evaluation (E6Q2.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

#define MAX 100

// Stack for operands
double valStack[MAX];
int valTop = -1;

void pushVal(double val) {
    if (valTop < MAX - 1) {
        valStack[++valTop] = val;
    }
}

double popVal() {
    if (valTop >= 0) {
        return valStack[valTop--];
    }
    printf("Error: Stack underflow during evaluation!\n");
    return 0;
}

// Evaluate postfix expression where tokens can be separated by spaces
double evaluatePostfix(const char* exp) {
    valTop = -1;
    char buffer[256];
    strncpy(buffer, exp, sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    char* token = strtok(buffer, " \t\r\n");
    while (token != NULL) {
        // If token is an operator and length is 1 (or handle negative numbers)
        if (strlen(token) == 1 && (token[0] == '+' || token[0] == '-' || token[0] == '*' ||
token[0] == '/' || token[0] == '^')) {
            double op2 = popVal();
            double op1 = popVal();
            double res = 0;

            switch (token[0]) {
                case '+': res = op1 + op2; break;
                case '-': res = op1 - op2; break;
                case '*': res = op1 * op2; break;
                case '/':
                    if (op2 == 0) {
                        printf("Error: Division by zero!\n");
                        return 0;
                    }
                    res = op1 / op2;
                    break;
                case '^': res = pow(op1, op2); break;
            }
            pushVal(res);
```

```
        } else {
            // Token is a number
            double num = atof(token);
            pushVal(num);
        }
        token = strtok(NULL, " \t\r\n");
    }

    return popVal();
}

int main() {
    printHeader();

    // Problem 2(a)
    const char* exp1 = "2 3 1 * + 9 -";
    printf("--- Case 1 (Manual Problem 2a) ---\n");
    printf("Expression: %s\n", exp1);
    double res1 = evaluatePostfix(exp1);
    printf("Result    : %.2f (Expected: -4)\n", res1);

    // Problem 2(b)
    const char* exp2 = "2 2 + 2 / 5 * 7 +";
    printf("\n--- Case 2 (Manual Problem 2b) ---\n");
    printf("Expression: %s\n", exp2);
    double res2 = evaluatePostfix(exp2);
    printf("Result    : %.2f (Expected: 17)\n", res2);

    // Interactive Case
    printf("\n--- Case 3 (Interactive Postfix Evaluation) ---\n");
    printf("Enter space-separated postfix expression (e.g. 10 5 + 3 *): ");
    char userExp[256];
    if (fgets(userExp, sizeof(userExp), stdin) != NULL) {
        userExp[strcspn(userExp, "\r\n")] = '\0';
        if (strlen(userExp) > 0) {
            double resUser = evaluatePostfix(userExp);
            printf("Result    : %.2f\n", resUser);
        }
    }

    printf("============================================================\n");
    return 0;
}
```

Figure 6.2: Terminal output for Postfix Expression Evaluation (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Case 1 (Manual Problem 2a) ---
Expression: 2 3 1 * + 9 -
Result    : -4.00 (Expected: -4)

--- Case 2 (Manual Problem 2b) ---
Expression: 2 2 + 2 / 5 * 7 +
Result    : 17.00 (Expected: 17)
```

Program 6.3: Balanced Parentheses Syntax Checker (E6Q3.c)

```
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

#define MAX 200

char stack[MAX];
int top = -1;

void push(char c) {
    if (top < MAX - 1) {
        stack[++top] = c;
    }
}

char pop() {
    if (top >= 0) {
        return stack[top--];
    }
    return '\0';
}

bool isEmpty() {
    return top == -1;
}

// Check if open and close brackets match
bool isMatchingPair(char character1, char character2) {
    if (character1 == '(' && character2 == ')') return true;
    if (character1 == '{' && character2 == '}') return true;
    if (character1 == '[' && character2 == ']') return true;
    return false;
}

// Function to check if expression has balanced brackets
bool areBracketsBalanced(const char* exp) {
    top = -1; // reset stack

    for (int i = 0; exp[i] != '\0'; i++) {
        char ch = exp[i];

        // If opening bracket, push to stack
        if (ch == '{' || ch == '(' || ch == '[') {
            push(ch);
        }
        // If closing bracket
        else if (ch == '}' || ch == ')' || ch == ']') {
            if (isEmpty()) {
                return false; // unmatched closing bracket
            }
            char openBracket = pop();
            if (!isMatchingPair(openBracket, ch)) {
                return false; // mismatched pair
            }
        }
    }
```

```
    // If stack is empty, all brackets were matched
    return isEmpty();
}

int main() {
    printHeader();

    // Test cases from the Lab Manual
    const char* test1 = "[( )]{ }{[( )( )]( )}";
    const char* test2 = "[( ])";

    printf("--- Test Case 1 (From Manual) ---\n");
    printf("Expression: %s\n", test1);
    printf("Result    : %s\n", areBracketsBalanced(test1) ? "Balanced" : "Not Balanced");

    printf("\n--- Test Case 2 (From Manual) ---\n");
    printf("Expression: %s\n", test2);
    printf("Result    : %s\n", areBracketsBalanced(test2) ? "Balanced" : "Not Balanced");

    // Interactive Test Case
    printf("\n--- Test Case 3 (Interactive Input) ---\n");
    char userExp[MAX];
    printf("Enter an expression to check: ");
    if (fgets(userExp, sizeof(userExp), stdin) != NULL) {
        userExp[strcspn(userExp, "\r\n")] = '\0';
        if (strlen(userExp) > 0) {
            printf("Expression: %s\n", userExp);
            printf("Result    : %s\n", areBracketsBalanced(userExp) ? "Balanced" : "Not
Balanced");
        }
    }

    printf("============================================================\n");
    return 0;
}
```

Figure 6.3: Terminal output for Balanced Parentheses Validation (25BCP113)

```
==================================================
  Name    : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Test Case 1 (From Manual) ---
Expression: [( )]{ }{[( )( )]( )}
Result    : Balanced

--- Test Case 2 (From Manual) ---
Expression: [( ])
Result    : Not Balanced

--- Test Case 3 (Interactive Input) ---
Enter an expression to check: (A+B)*(C-D)
Expression: (A+B)*(C-D)
Result    : Balanced
============================================================
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• Postfix notation removes ambiguous parentheses and precedence rules, allowing single-scan linear O(N) evaluation.

• The Infix-to-Postfix conversion correctly respects right-associativity of exponents (^) and left-associativity of *, /, +, -.

• Bracket validation matches LIFO closing brackets in O(N) time and O(N) space.

EXPERIMENT NO. 7

Queue (Array Queue, Linked List Queue, Priority Queue & Deque Operations)

1. Objective

To study First-In-First-Out (FIFO) Queues implemented via linear/circular arrays and dynamic linked lists, Priority Queues with priority-ordered insertion, and Double-Ended Queues (Deque) supporting dual-end insertion, deletion, and state verification.

2. Problem Statements / Program Definitions

1. Implement various functionalities of Queue using Arrays (insertion, deletion, front element, rear element etc.).

2. Implement various functionalities of Queue using Linked Lists.

3. Implement Priority Queue, where every element has a priority associated with it. Perform operations like Insertion and Deletion in a priority queue.

4. Implement Double Ended Queue that supports: insertFront(), insertLast(), deleteFront(), deleteLast().

5. Implement Double Ended Queue that supports: getFront(), getRear(), isEmpty(), isFull().

3. Implementation & Output Snapshots

Program 7.1: Queue Implementation using Array (E7Q1.c)

```
#include <stdio.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

#define SIZE 5

int queue[SIZE];
int front = -1;
int rear = -1;

bool isFull() {
    return rear == SIZE - 1;
}

bool isEmpty() {
    return (front == -1 || front > rear);
}

void enqueue(int val) {
    if (isFull()) {
        printf(">> [Overflow] Queue is full! Cannot insert %d.\n", val);
        return;
    }
    if (front == -1) {
        front = 0;
    }
    queue[++rear] = val;
    printf(">> Enqueued: %d (Front: %d, Rear: %d)\n", val, front, rear);
}
```

```
int dequeue() {
    if (isEmpty()) {
        printf(">> [Underflow] Queue is empty!\n");
        return -1;
    }
    int val = queue[front++];
    printf(">> Dequeued: %d\n", val);
    if (front > rear) {
        front = rear = -1; // Reset queue
    }
    return val;
}

int getFront() {
    if (isEmpty()) {
        printf(">> Queue is empty!\n");
        return -1;
    }
    return queue[front];
}

int getRear() {
    if (isEmpty()) {
        printf(">> Queue is empty!\n");
        return -1;
    }
    return queue[rear];
}

void display() {
    if (isEmpty()) {
        printf("Queue State: [Empty]\n");
        return;
    }
    printf("Queue State: Front -> ");
    for (int i = front; i <= rear; i++) {
        printf("[%d] ", queue[i]);
    }
    printf("<- Rear\n");
}

int main() {
    printHeader();

    printf("--- Performing Queue Operations ---\n");
    enqueue(10);
    enqueue(20);
    enqueue(30);
    enqueue(40);
    enqueue(50);
    enqueue(60); // Test Overflow
    display();

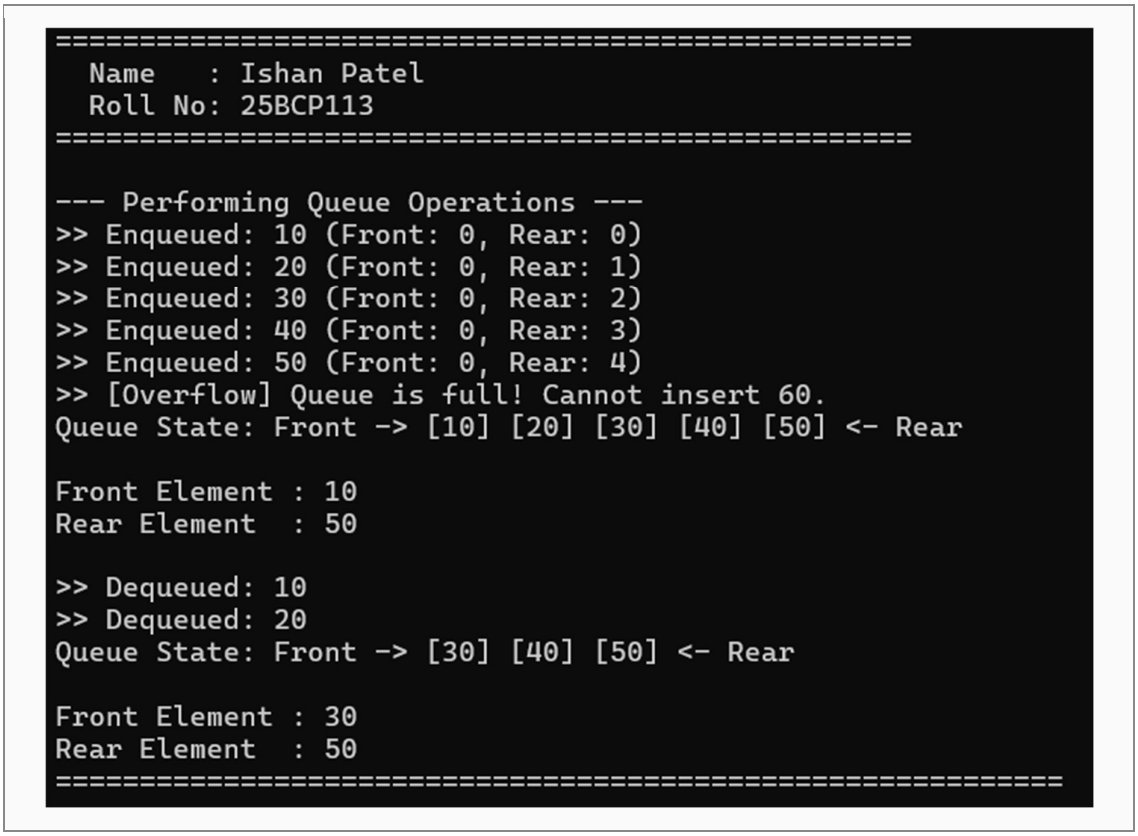
    printf("\nFront Element : %d\n", getFront());
    printf("Rear Element  : %d\n\n", getRear());

    dequeue();
    dequeue();
    display();

    printf("\nFront Element : %d\n", getFront());
    printf("Rear Element  : %d\n", getRear());
    printf("============================================================\n");

    return 0;
}
```

Figure 7.1: Terminal output for Queue operations using Array (25BCP113)



Program 7.2: Queue Implementation using Linked List (E7Q2.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct Node {
    int data;
    struct Node* next;
};

struct Queue {
    struct Node *front;
    struct Node *rear;
};

void initQueue(struct Queue* q) {
    q->front = NULL;
    q->rear = NULL;
}

bool isEmpty(struct Queue* q) {
```

```
    return q->front == NULL;
}

void enqueue(struct Queue* q, int val) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf(">> [Heap Overflow] Memory allocation failed!\n");
        return;
    }
    newNode->data = val;
    newNode->next = NULL;

    if (q->rear == NULL) {
        q->front = q->rear = newNode;
    } else {
        q->rear->next = newNode;
        q->rear = newNode;
    }
    printf(">> Enqueued [%d] to Linked List Queue\n", val);
}

int dequeue(struct Queue* q) {
    if (isEmpty(q)) {
        printf(">> [Underflow] Queue is empty!\n");
        return -1;
    }

    struct Node* temp = q->front;
    int val = temp->data;
    q->front = q->front->next;

    if (q->front == NULL) {
        q->rear = NULL;
    }
    free(temp);
    printf(">> Dequeued [%d]\n", val);
    return val;
}

int getFront(struct Queue* q) {
    if (isEmpty(q)) {
        printf(">> Queue is empty!\n");
        return -1;
    }
    return q->front->data;
}

int getRear(struct Queue* q) {
    if (isEmpty(q)) {
        printf(">> Queue is empty!\n");
        return -1;
    }
    return q->rear->data;
}

void display(struct Queue* q) {
    if (isEmpty(q)) {
        printf("Queue State: [Empty]\n");
        return;
    }
    printf("Queue State: Front -> ");
    struct Node* curr = q->front;
    while (curr != NULL) {
        printf("[%d]", curr->data);
        if (curr->next != NULL) printf(" -> ");
        curr = curr->next;
```

```
    }
    printf(" <- Rear\n");
}

void freeQueue(struct Queue* q) {
    while (!isEmpty(q)) {
        dequeue(q);
    }
}

int main() {
    printHeader();

    struct Queue q;
    initQueue(&q);

    printf("--- Performing Linked List Queue Operations ---\n");
    enqueue(&q, 100);
    enqueue(&q, 200);
    enqueue(&q, 300);
    enqueue(&q, 400);
    display(&q);

    printf("\nFront Element : %d\n", getFront(&q));
    printf("Rear Element  : %d\n\n", getRear(&q));

    dequeue(&q);
    dequeue(&q);
    display(&q);

    printf("\nFront Element : %d\n", getFront(&q));
    printf("Rear Element  : %d\n", getRear(&q));
    printf("============================================================\n");

    freeQueue(&q);
    return 0;
}
```

Figure 7.2: Terminal output for Queue operations using Linked List (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Performing Linked List Queue Operations ---
>> Enqueued [100] to Linked List Queue
>> Enqueued [200] to Linked List Queue
>> Enqueued [300] to Linked List Queue
>> Enqueued [400] to Linked List Queue
Queue State: Front -> [100] -> [200] -> [300] -> [400] <- Rear

Front Element : 100
Rear Element  : 400

>> Dequeued [100]
>> Dequeued [200]
Queue State: Front -> [300] -> [400] <- Rear

Front Element : 300
Rear Element  : 400
============================================================
>> Dequeued [300]
>> Dequeued [400]
```

Program 7.3: Priority Queue Implementation (E7Q3.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("=================================================\n");
    printf("  Name   : Ishan Patel                           \n");
    printf("  Roll No: 25BCP113                              \n");
    printf("=================================================\n\n");
}

// Node represents an element with a value and an associated priority
// (Higher priority number indicates higher service priority)
struct PQNode {
    int data;
    int priority;
    struct PQNode* next;
};

struct PQNode* createNode(int data, int priority) {
    struct PQNode* newNode = (struct PQNode*)malloc(sizeof(struct PQNode));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = data;
    newNode->priority = priority;
    newNode->next = NULL;
    return newNode;
}

bool isEmpty(struct PQNode* head) {
    return head == NULL;
}

// Enqueue element according to priority (descending order of priority)
void enqueue(struct PQNode** head, int data, int priority) {
    struct PQNode* newNode = createNode(data, priority);

    // If list is empty or new node has higher priority than head
    if (*head == NULL || priority > (*head)->priority) {
        newNode->next = *head;
        *head = newNode;
        printf(">> Enqueued [%d] (Priority: %d) at Front\n", data, priority);
        return;
    }

    // Traverse list to find position
    struct PQNode* curr = *head;
    while (curr->next != NULL && curr->next->priority >= priority) {
        curr = curr->next;
    }

    newNode->next = curr->next;
    curr->next = newNode;
    printf(">> Enqueued [%d] (Priority: %d)\n", data, priority);
}

// Dequeue element with highest priority (always at head)
int dequeue(struct PQNode** head) {
    if (isEmpty(*head)) {
        printf(">> [Underflow] Priority Queue is empty!\n");
```

```
        return -1;
    }

    struct PQNode* temp = *head;
    int val = temp->data;
    int p = temp->priority;
    *head = (*head)->next;
    free(temp);
    printf(">> Dequeued Highest Priority Element: [%d] (Priority: %d)\n", val, p);
    return val;
}

// Peek highest priority element
void peek(struct PQNode* head) {
    if (isEmpty(head)) {
        printf(">> Priority Queue is empty!\n");
        return;
    }
    printf(">> Highest Priority Element on Top: [%d] (Priority: %d)\n", head->data, head-
>priority);
}

// Display Priority Queue
void display(struct PQNode* head) {
    if (isEmpty(head)) {
        printf("Priority Queue: [Empty]\n");
        return;
    }

    printf("Priority Queue: Top -> ");
    struct PQNode* curr = head;
    while (curr != NULL) {
        printf("[Val: %d | Pri: %d]", curr->data, curr->priority);
        if (curr->next != NULL) printf(" -> ");
        curr = curr->next;
    }
    printf(" -> End\n");
}

void freePQ(struct PQNode** head) {
    while (!isEmpty(*head)) {
        dequeue(head);
    }
}

int main() {
    printHeader();

    struct PQNode* pq = NULL;

    printf("--- Enqueueing Elements with (Value, Priority) ---\n");
    enqueue(&pq, 10, 1);
    enqueue(&pq, 40, 4);
    enqueue(&pq, 20, 2);
    enqueue(&pq, 50, 5);
    enqueue(&pq, 30, 3);

    printf("\n--- Display Priority Queue (Ordered by Priority) ---\n");
    display(pq);

    printf("\n--- Peeking Highest Priority ---\n");
    peek(pq);

    printf("\n--- Dequeue Operations (Highest Priority Removed First) ---\n");
    dequeue(&pq);
    dequeue(&pq);
```

```
    display(pq);

    printf("============================================================\n");
    freePQ(&pq);
    return 0;
}
```

Figure 7.3: Terminal output for Priority Queue operations (25BCP113)

```
================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
================================================

--- Enqueueing Elements with (Value, Priority) ---
>> Enqueued [10] (Priority: 1) at Front
>> Enqueued [40] (Priority: 4) at Front
>> Enqueued [20] (Priority: 2)
>> Enqueued [50] (Priority: 5) at Front
>> Enqueued [30] (Priority: 3)

--- Display Priority Queue (Ordered by Priority) ---
Priority Queue: Top -> [Val: 50 | Pri: 5] -> [Val: 40 | Pri: 4] -> [Val: 30 | Pri: 3] -> [Val: 20 | Pri: 2] -> [Val: 10 | Pri: 1] -> End

--- Peeking Highest Priority ---
>> Highest Priority Element on Top: [50] (Priority: 5)

--- Dequeue Operations (Highest Priority Removed First) ---
>> Dequeued Highest Priority Element: [50] (Priority: 5)
>> Dequeued Highest Priority Element: [40] (Priority: 4)
Priority Queue: Top -> [Val: 30 | Pri: 3] -> [Val: 20 | Pri: 2] -> [Val: 10 | Pri: 1] -> End
============================================================
>> Dequeued Highest Priority Element: [30] (Priority: 3)
>> Dequeued Highest Priority Element: [20] (Priority: 2)
>> Dequeued Highest Priority Element: [10] (Priority: 1)
```

Program 7.4: Deque Operations (insertFront, insertLast, deleteFront, deleteLast) (E7Q4.c)

```
#include <stdio.h>
#include <stdbool.h>

void printHeader() {
    printf("====================================================\n");
    printf("  Name   : Ishan Patel                              \n");
    printf("  Roll No: 25BCP113                                 \n");
    printf("====================================================\n\n");
}

#define SIZE 5

int deque[SIZE];
int front = -1;
int rear = -1;

bool isFull() {
    return ((front == 0 && rear == SIZE - 1) || (front == rear + 1));
}

bool isEmpty() {
    return (front == -1);
}

// a. insertFront(): Adds an item at the front of Deque
void insertFront(int val) {
    if (isFull()) {
        printf(">> [Overflow] Deque is Full! Cannot insert %d at front.\n", val);
        return;
    }
    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (front == 0) {
```

```
        front = SIZE - 1;
    } else {
        front--;
    }
    deque[front] = val;
    printf(">> Inserted [%d] at Front. (Front: %d, Rear: %d)\n", val, front, rear);
}

// b. insertLast(): Adds an item at the rear of Deque
void insertLast(int val) {
    if (isFull()) {
        printf(">> [Overflow] Deque is Full! Cannot insert %d at rear.\n", val);
        return;
    }
    if (front == -1) {
        front = 0;
        rear = 0;
    } else if (rear == SIZE - 1) {
        rear = 0;
    } else {
        rear++;
    }
    deque[rear] = val;
    printf(">> Inserted [%d] at Rear.  (Front: %d, Rear: %d)\n", val, front, rear);
}

// c. deleteFront(): Deletes an item from the front of Deque
void deleteFront() {
    if (isEmpty()) {
        printf(">> [Underflow] Deque is Empty! Cannot delete from front.\n");
        return;
    }
    int val = deque[front];
    printf(">> Deleted [%d] from Front.\n", val);

    if (front == rear) {
        front = -1;
        rear = -1;
    } else if (front == SIZE - 1) {
        front = 0;
    } else {
        front++;
    }
}

// d. deleteLast(): Deletes an item from the rear of Deque
void deleteLast() {
    if (isEmpty()) {
        printf(">> [Underflow] Deque is Empty! Cannot delete from rear.\n");
        return;
    }
    int val = deque[rear];
    printf(">> Deleted [%d] from Rear.\n", val);

    if (front == rear) {
        front = -1;
        rear = -1;
    } else if (rear == 0) {
        rear = SIZE - 1;
    } else {
        rear--;
    }
}

void display() {
    if (isEmpty()) {
        printf("Deque State: [Empty]\n");
```

```
        return;
    }
    printf("Deque State: Front -> ");
    int i = front;
    while (1) {
        printf("[%d] ", deque[i]);
        if (i == rear) break;
        i = (i + 1) % SIZE;
    }
    printf("<- Rear\n");
}

int main() {
    printHeader();

    printf("--- 1. Testing insertFront and insertLast ---\n");
    insertLast(10);
    insertLast(20);
    insertFront(5);
    insertLast(30);
    insertFront(1);
    display();

    printf("\n--- 2. Testing Overflow ---\n");
    insertLast(99);

    printf("\n--- 3. Testing deleteFront and deleteLast ---\n");
    deleteFront();
    display();
    deleteLast();
    display();

    printf("============================================================\n");
    return 0;
}
```

Figure 7.4: Terminal output for Double Ended Queue insertion and deletion (25BCP113)

```
==================================================
  Name    : Ishan Patel
  Roll No: 25BCP113
==================================================

--- 1. Testing insertFront and insertLast ---
>> Inserted [10] at Rear.  (Front: 0, Rear: 0)
>> Inserted [20] at Rear.  (Front: 0, Rear: 1)
>> Inserted [5] at Front. (Front: 4, Rear: 1)
>> Inserted [30] at Rear.  (Front: 4, Rear: 2)
>> Inserted [1] at Front. (Front: 3, Rear: 2)
Deque State: Front -> [1] [5] [10] [20] [30] <- Rear

--- 2. Testing Overflow ---
>> [Overflow] Deque is Full! Cannot insert 99 at rear.

--- 3. Testing deleteFront and deleteLast ---
>> Deleted [1] from Front.
Deque State: Front -> [5] [10] [20] [30] <- Rear
>> Deleted [30] from Rear.
Deque State: Front -> [5] [10] [20] <- Rear
============================================================
```

Program 7.5: Deque Inspection (getFront, getRear, isEmpty, isFull) (E7Q5.c)

```
#include <stdio.h>
#include <stdbool.h>

void printHeader() {
    printf("=================================================\n");
    printf("  Name   : Ishan Patel                           \n");
    printf("  Roll No: 25BCP113                              \n");
    printf("=================================================\n\n");
}

#define SIZE 5

int deque[SIZE];
int front = -1;
int rear = -1;

// c. isEmpty(): Checks whether Deque is empty or not
bool isEmpty() {
    return (front == -1);
}

// d. isFull(): Checks whether Deque is full or not
bool isFull() {
    return ((front == 0 && rear == SIZE - 1) || (front == rear + 1));
}

// a. getFront(): Gets the front item from the queue
int getFront() {
    if (isEmpty()) {
        printf(">> Deque is Empty!\n");
        return -1;
    }
    return deque[front];
}

// b. getRear(): Gets the last item from queue
int getRear() {
    if (isEmpty()) {
        printf(">> Deque is Empty!\n");
        return -1;
    }
    return deque[rear];
}

void insertRear(int val) {
    if (isFull()) {
        printf(">> [Overflow] Deque is Full!\n");
        return;
    }
    if (front == -1) {
        front = rear = 0;
    } else if (rear == SIZE - 1) {
        rear = 0;
    } else {
        rear++;
    }
    deque[rear] = val;
    printf(">> Inserted [%d] at Rear\n", val);
}

void display() {
    if (isEmpty()) {
```

```
        printf("Deque State: [Empty]\n");
        return;
    }
    printf("Deque Elements: ");
    int i = front;
    while (1) {
        printf("[%d] ", deque[i]);
        if (i == rear) break;
        i = (i + 1) % SIZE;
    }
    printf("\n");
}

int main() {
    printHeader();

    printf("--- Initial State Check ---\n");
    printf("  isEmpty() : %s\n", isEmpty() ? "True" : "False");
    printf("  isFull()  : %s\n\n", isFull() ? "True" : "False");

    printf("--- Inserting 5 Elements to Fill Deque ---\n");
    insertRear(10);
    insertRear(20);
    insertRear(30);
    insertRear(40);
    insertRear(50);
    display();

    printf("\n--- State Check when Full ---\n");
    printf("  isEmpty()  : %s\n", isEmpty() ? "True" : "False");
    printf("  isFull()   : %s\n", isFull() ? "True" : "False");
    printf("  getFront() : %d\n", getFront());
    printf("  getRear()  : %d\n", getRear());
    printf("============================================================\n");

    return 0;
}
```

Figure 7.5: Terminal output for Double Ended Queue state inspection (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Initial State Check ---
  isEmpty() : True
  isFull()  : False

--- Inserting 5 Elements to Fill Deque ---
>> Inserted [10] at Rear
>> Inserted [20] at Rear
>> Inserted [30] at Rear
>> Inserted [40] at Rear
>> Inserted [50] at Rear
Deque Elements: [10] [20] [30] [40] [50]

--- State Check when Full ---
  isEmpty()  : False
  isFull()   : True
  getFront() : 10
  getRear()  : 50
============================================================
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

- Standard Queues enforce FIFO ordering: enqueue at rear, dequeue at front in O(1) time.
- Circular array queues prevent false overflow by wrapping pointers modulo array capacity.
- Priority Queues maintain sorted priority order during insertion (O(N)), enabling O(1) highest-priority extraction.
- Deques provide maximum queuing versatility with O(1) push and pop operations at both ends.

EXPERIMENT NO. 8

Trees (BST Insert/Search/Delete, Tree Traversals & Preorder BST Construction)

1. Objective

To design non-linear hierarchical tree structures: Binary Search Tree (BST) insertion, search, and 3-case deletion; Depth-First traversals (Pre-order, In-order, Post-order); and reconstructing a BST from its pre-order traversal sequence in optimal O(N) time.

2. Problem Statements / Program Definitions

1. Write a program to insert an element, delete an element and search an element in the Binary Tree (BST).

2. Study and implement the Binary Tree and perform following three types of traversals in a binary tree: (a) Pre-order, (b) In-order, (c) Post-order.

3. Given a preorder traversal sequence of a Binary Search Tree, construct the corresponding Binary Search Tree.

3. Implementation & Output Snapshots

Program 8.1: BST Insert, Search, and 3-Case Deletion (E8Q1.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Create a new BST node
struct Node* createNode(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Insert an element into BST
struct Node* insert(struct Node* root, int value) {
    if (root == NULL) {
        return createNode(value);
    }
    if (value < root->data) {
        root->left = insert(root->left, value);
    } else if (value > root->data) {
        root->right = insert(root->right, value);
    } else {
```

```
        printf("  [Notice] Duplicate key %d ignored.\n", value);
    }
    return root;
}

// Search for an element in BST
bool search(struct Node* root, int key) {
    if (root == NULL) {
        return false;
    }
    if (root->data == key) {
        return true;
    }
    if (key < root->data) {
        return search(root->left, key);
    }
    return search(root->right, key);
}

// Find minimum value node in a subtree (used for deletion with two children)
struct Node* findMin(struct Node* node) {
    struct Node* current = node;
    while (current && current->left != NULL) {
        current = current->left;
    }
    return current;
}

// Delete an element from BST
struct Node* deleteNode(struct Node* root, int key) {
    if (root == NULL) {
        printf("  [Notice] Key %d not found in tree.\n", key);
        return root;
    }

    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    } else {
        // Node found: handle 3 cases

        // Case 1 & 2: Node with 0 or 1 child
        if (root->left == NULL) {
            struct Node* temp = root->right;
            free(root);
            return temp;
        } else if (root->right == NULL) {
            struct Node* temp = root->left;
            free(root);
            return temp;
        }

        // Case 3: Node with 2 children
        // Get the inorder successor (smallest in the right subtree)
        struct Node* temp = findMin(root->right);
        root->data = temp->data; // Copy successor's content to this node
        root->right = deleteNode(root->right, temp->data); // Delete successor
    }
    return root;
}

// Inorder traversal to verify BST structure (should be sorted)
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
```

```
        inorder(root->right);
    }
}

void freeTree(struct Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main() {
    printHeader();

    struct Node* root = NULL;

    // 1. Insertion
    printf("--- 1. Inserting Elements: 50, 30, 20, 40, 70, 60, 80 ---\n");
    int elements[] = {50, 30, 20, 40, 70, 60, 80};
    int n = sizeof(elements) / sizeof(elements[0]);

    for (int i = 0; i < n; i++) {
        root = insert(root, elements[i]);
    }

    printf("Inorder Traversal (Sorted): ");
    inorder(root);
    printf("\n\n");

    // 2. Searching
    printf("--- 2. Searching Elements ---\n");
    int keysToSearch[] = {40, 99};
    for (int i = 0; i < 2; i++) {
        int k = keysToSearch[i];
        if (search(root, k)) {
            printf("  -> Element %d is PRESENT in the BST.\n", k);
        } else {
            printf("  -> Element %d is NOT PRESENT in the BST.\n", k);
        }
    }

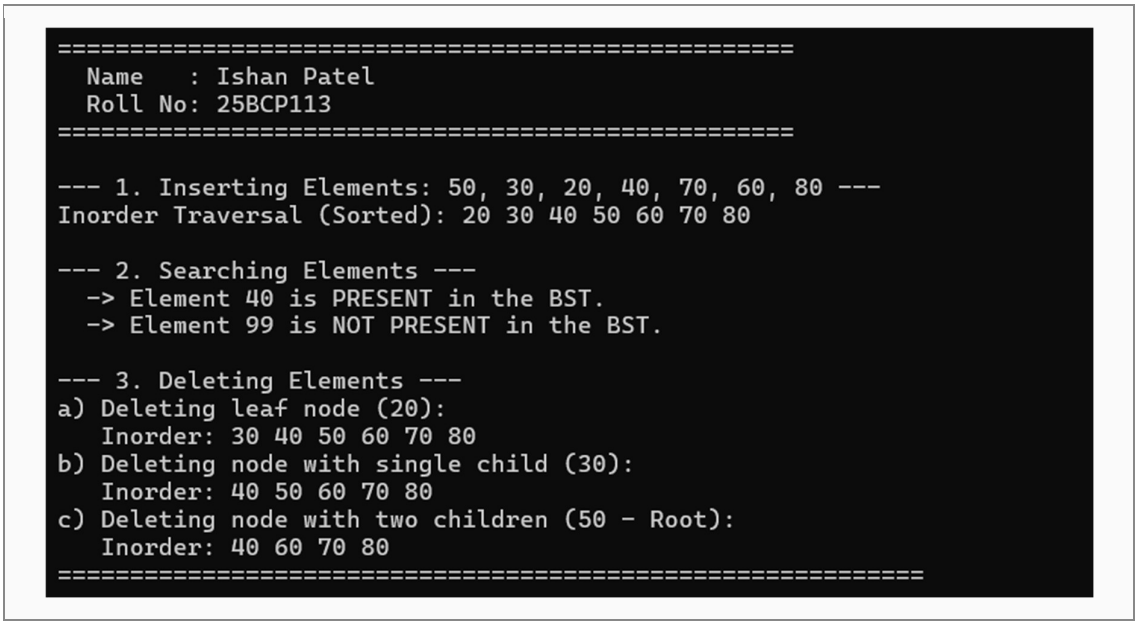
    // 3. Deletion
    printf("\n--- 3. Deleting Elements ---\n");
    printf("a) Deleting leaf node (20):\n");
    root = deleteNode(root, 20);
    printf("   Inorder: "); inorder(root); printf("\n");

    printf("b) Deleting node with single child (30):\n");
    root = deleteNode(root, 30);
    printf("   Inorder: "); inorder(root); printf("\n");

    printf("c) Deleting node with two children (50 - Root):\n");
    root = deleteNode(root, 50);
    printf("   Inorder: "); inorder(root); printf("\n");

    printf("============================================================\n");
    freeTree(root);
    return 0;
}
```

Figure 8.1: Terminal output for BST Insert, Search, and Deletion (25BCP113)



```
==========================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==========================================================

--- 1. Inserting Elements: 50, 30, 20, 40, 70, 60, 80 ---
Inorder Traversal (Sorted): 20 30 40 50 60 70 80

--- 2. Searching Elements ---
  -> Element 40 is PRESENT in the BST.
  -> Element 99 is NOT PRESENT in the BST.

--- 3. Deleting Elements ---
a) Deleting leaf node (20):
   Inorder: 30 40 50 60 70 80
b) Deleting node with single child (30):
   Inorder: 40 50 60 70 80
c) Deleting node with two children (50 - Root):
   Inorder: 40 60 70 80
==========================================================
```

Program 8.2: Binary Tree Traversals (Pre-order, In-order, Post-order) (E8Q2.c)

```c
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("==========================================================\n");
    printf("  Name   : Ishan Patel                                    \n");
    printf("  Roll No: 25BCP113                                       \n");
    printf("==========================================================\n\n");
}

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Create a new binary tree node
struct Node* newNode(int data) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    if (node == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

// a. Pre-order Traversal: Root -> Left -> Right
void printPreorder(struct Node* node) {
    if (node == NULL) return;
    printf("%d ", node->data);
    printPreorder(node->left);
    printPreorder(node->right);
}
```

```
// b. In-order Traversal: Left -> Root -> Right
void printInorder(struct Node* node) {
    if (node == NULL) return;
    printInorder(node->left);
    printf("%d ", node->data);
    printInorder(node->right);
}

// c. Post-order Traversal: Left -> Right -> Root
void printPostorder(struct Node* node) {
    if (node == NULL) return;
    printPostorder(node->left);
    printPostorder(node->right);
    printf("%d ", node->data);
}

void freeTree(struct Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main() {
    printHeader();

    /*
            Constructing following Binary Tree:
                        1
                      /   \
                     2     3
                    / \   / \
                   4   5 6   7
    */
    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->right->left = newNode(6);
    root->right->right = newNode(7);

    printf("--- Tree Structure Constructed ---\n");
    printf("             1          \n");
    printf("           /   \\       \n");
    printf("          2     3       \n");
    printf("         / \\   / \\     \n");
    printf("        4   5 6   7     \n");
    printf("------------------------------------------------------------\n");

    printf("\n(a) Pre-order Traversal  [Root -> Left -> Right]:\n    ");
    printPreorder(root);
    printf("\n");

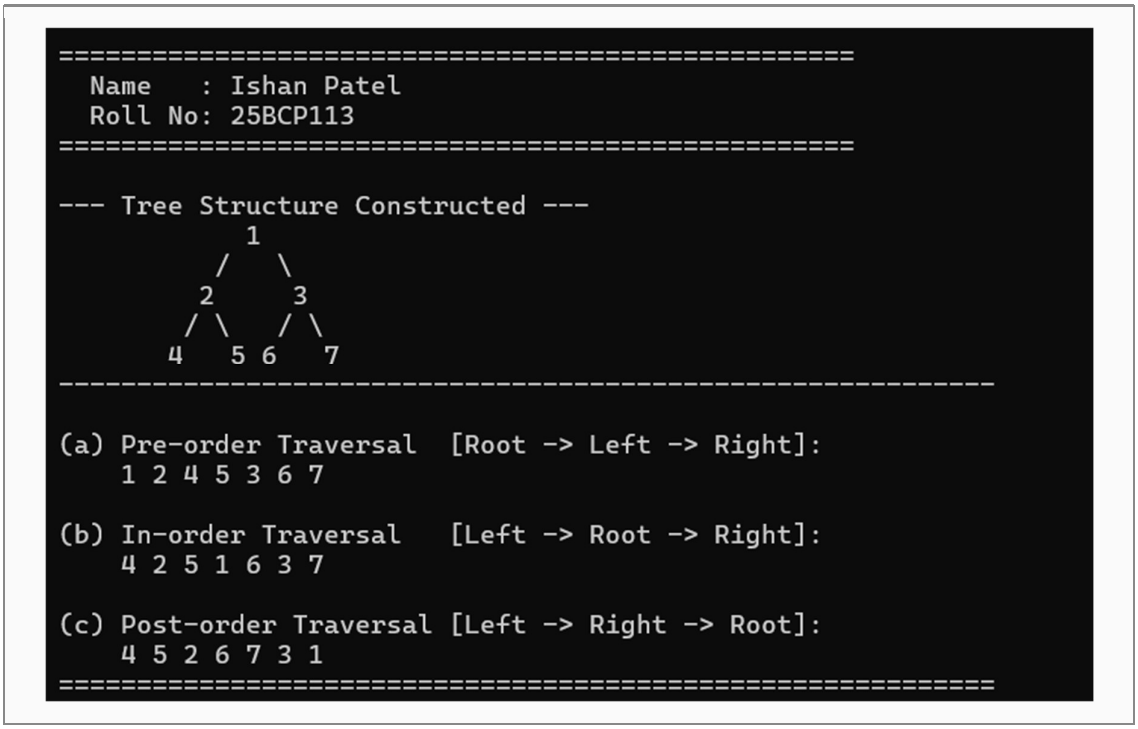
    printf("\n(b) In-order Traversal   [Left -> Root -> Right]:\n    ");
    printInorder(root);
    printf("\n");

    printf("\n(c) Post-order Traversal [Left -> Right -> Root]:\n    ");
    printPostorder(root);
    printf("\n");

    printf("============================================================\n");
    freeTree(root);
    return 0;
```

```
}
```

Figure 8.2: Terminal output for Binary Tree Traversals (25BCP113)



Program 8.3: Construct BST from Preorder Traversal Sequence (E8Q3.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int data) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    if (temp == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    temp->data = data;
    temp->left = temp->right = NULL;
    return temp;
}

// O(N) recursive function to construct BST from preorder traversal
```

```
// using value range constraints (min, max)
struct Node* constructBSTUtil(int pre[], int* preIndex, int key, int min, int max, int size)
{
    if (*preIndex >= size) {
        return NULL;
    }

    struct Node* root = NULL;

    // If current element is within range, it belongs to this subtree
    if (key > min && key < max) {
        root = newNode(key);
        *preIndex = *preIndex + 1;

        if (*preIndex < size) {
            // All elements in left subtree must be < key
            root->left = constructBSTUtil(pre, preIndex, pre[*preIndex], min, key, size);
        }
        if (*preIndex < size) {
            // All elements in right subtree must be > key
            root->right = constructBSTUtil(pre, preIndex, pre[*preIndex], key, max, size);
        }
    }

    return root;
}

struct Node* constructBST(int pre[], int size) {
    int preIndex = 0;
    return constructBSTUtil(pre, &preIndex, pre[0], INT_MIN, INT_MAX, size);
}

void printInorder(struct Node* node) {
    if (node == NULL) return;
    printInorder(node->left);
    printf("%d ", node->data);
    printInorder(node->right);
}

void printPreorder(struct Node* node) {
    if (node == NULL) return;
    printf("%d ", node->data);
    printPreorder(node->left);
    printPreorder(node->right);
}

void freeTree(struct Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
        free(root);
    }
}

int main() {
    printHeader();

    // Example preorder sequence
    int pre[] = {10, 5, 1, 7, 40, 50};
    int size = sizeof(pre) / sizeof(pre[0]);

    printf("Input Preorder Sequence: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", pre[i]);
    }
    printf("\n-------------------------------------------------------------\n");
```

```
    printf("Constructing Binary Search Tree...\n");
    struct Node* root = constructBST(pre, size);
    printf(">> BST successfully constructed!\n\n");

    printf("Verification Traversal Results:\n");
    printf("  1. Inorder Traversal (Must be strictly sorted): \n     ");
    printInorder(root);
    printf("\n\n");

    printf("  2. Preorder of Constructed BST (Matches input): \n     ");
    printPreorder(root);
    printf("\n");

    printf("============================================================\n");
    freeTree(root);
    return 0;
}
```

Figure 8.3: Terminal output for BST Construction from Preorder sequence (25BCP113)

```
=====================================================
  Name    : Ishan Patel
  Roll No: 25BCP113
=====================================================

Input Preorder Sequence: 10 5 1 7 40 50
------------------------------------------------------------
Constructing Binary Search Tree...
>> BST successfully constructed!

Verification Traversal Results:
  1. Inorder Traversal (Must be strictly sorted):
     1 5 7 10 40 50

  2. Preorder of Constructed BST (Matches input):
     10 5 1 7 40 50
============================================================
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• BST property (Left < Root < Right) allows O(log N) average time for search, insertion, and deletion (degrading to O(N) in skewed cases).

• BST Deletion handles 3 cases: leaf node removal, single-child link replacement, and two-child node replacement with inorder successor.

• In-order traversal of a valid BST always outputs keys in strictly sorted ascending order.

• BST reconstruction from preorder runs in optimal O(N) time using recursive range bounds.

EXPERIMENT NO. 9

Graphs (Breadth First Search, Depth First Search, Cycle Detection & Kruskal's MST)

1. Objective

To study and implement fundamental graph traversal algorithms (BFS and DFS) with complexity analysis, cycle detection in directed and undirected graphs, and Minimum Spanning Tree (MST) construction using greedy Kruskal's algorithm with Disjoint Set Union (DSU).

2. Problem Statements / Program Definitions

1. For a given graph G = (V, E), study and implement Breadth First Search (BFS) and perform complexity analysis in terms of time and space.

2. For a given graph G = (V, E), study and implement Depth First Search (DFS) and perform complexity analysis in terms of time and space.

3. Given a directed graph, check whether the graph contains a cycle or not. Perform same task for undirected graph as well.

4. Implement Minimum Spanning Tree (MST) using the greedy Kruskal's algorithm.

3. Implementation & Output Snapshots

Program 9.1: Breadth First Search (BFS) with Complexity Analysis (E9Q1.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("=================================================\n");
    printf("  Name   : Ishan Patel                           \n");
    printf("  Roll No: 25BCP113                              \n");
    printf("=================================================\n\n");
}

// Graph Node for Adjacency List
struct AdjListNode {
    int dest;
    struct AdjListNode* next;
};

struct Graph {
    int V;
    struct AdjListNode** adjLists;
};

struct AdjListNode* createNode(int dest) {
    struct AdjListNode* newNode = (struct AdjListNode*)malloc(sizeof(struct AdjListNode));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = vertices;
    graph->adjLists = (struct AdjListNode**)malloc(vertices * sizeof(struct AdjListNode*));

    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
```

```
    }
    return graph;
}

// Add undirected edge
void addEdge(struct Graph* graph, int src, int dest) {
    // Edge from src to dest
    struct AdjListNode* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    // Edge from dest to src
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

// BFS using queue
void BFS(struct Graph* graph, int startVertex) {
    bool* visited = (bool*)calloc(graph->V, sizeof(bool));
    int* queue = (int*)malloc(graph->V * sizeof(int));
    int front = 0, rear = 0;

    visited[startVertex] = true;
    queue[rear++] = startVertex;

    printf("Breadth First Search Traversal starting from vertex %d:\n  -> ", startVertex);

    while (front < rear) {
        int currentVertex = queue[front++];
        printf("%d ", currentVertex);

        struct AdjListNode* temp = graph->adjLists[currentVertex];
        while (temp != NULL) {
            int adjVertex = temp->dest;
            if (!visited[adjVertex]) {
                visited[adjVertex] = true;
                queue[rear++] = adjVertex;
            }
            temp = temp->next;
        }
    }
    printf("\n");

    free(visited);
    free(queue);
}

void printComplexityAnalysis() {
    printf("\n================ COMPLEXITY ANALYSIS ================\n");
    printf(" 1. Time Complexity:\n");
    printf("    - Adjacency List: O(V + E)\n");
    printf("      Where V is the number of vertices and E is the number of edges.\n");
    printf("      Each vertex is enqueued once, and each edge is examined once.\n");
    printf(" 2. Space Complexity:\n");
    printf("    - O(V) for the visited boolean array and BFS queue.\n");
    printf("=====================================================\n");
}

void freeGraph(struct Graph* graph) {
    for (int i = 0; i < graph->V; i++) {
        struct AdjListNode* curr = graph->adjLists[i];
        while (curr != NULL) {
            struct AdjListNode* temp = curr;
            curr = curr->next;
            free(temp);
```

```
        }
    }
    free(graph->adjLists);
    free(graph);
}

int main() {
    printHeader();

    int V = 6;
    struct Graph* graph = createGraph(V);

    /*
          Graph Structure:
             0 --- 1
             |   / |
             |  /  |
             2 --- 3
             \    /
              4  5
    */
    addEdge(graph, 0, 1);
    addEdge(graph, 0, 2);
    addEdge(graph, 1, 2);
    addEdge(graph, 1, 3);
    addEdge(graph, 2, 3);
    addEdge(graph, 2, 4);
    addEdge(graph, 3, 5);

    printf("Graph with %d vertices constructed:\n", V);
    printf("Edges: (0-1), (0-2), (1-2), (1-3), (2-3), (2-4), (3-5)\n\n");

    BFS(graph, 0);

    printComplexityAnalysis();

    freeGraph(graph);
    return 0;
}
```

Figure 9.1: Terminal output for Breadth First Search Traversal (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

Graph with 6 vertices constructed:
Edges: (0-1), (0-2), (1-2), (1-3), (2-3), (2-4), (3-5)

Breadth First Search Traversal starting from vertex 0:
  -> 0 2 1 4 3 5

================ COMPLEXITY ANALYSIS ================
 1. Time Complexity:
    - Adjacency List: O(V + E)
      Where V is the number of vertices and E is the number of edges.
      Each vertex is enqueued once, and each edge is examined once.
 2. Space Complexity:
    - O(V) for the visited boolean array and BFS queue.
==================================================
```

Program 9.2: Depth First Search (DFS) with Complexity Analysis (E9Q2.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

// Graph Node for Adjacency List
struct AdjListNode {
    int dest;
    struct AdjListNode* next;
};

struct Graph {
    int V;
    struct AdjListNode** adjLists;
};

struct AdjListNode* createNode(int dest) {
    struct AdjListNode* newNode = (struct AdjListNode*)malloc(sizeof(struct AdjListNode));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

struct Graph* createGraph(int vertices) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = vertices;
    graph->adjLists = (struct AdjListNode**)malloc(vertices * sizeof(struct AdjListNode*));

    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
    }
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest) {
    // Edge from src to dest
    struct AdjListNode* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    // Edge from dest to src
    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}

// DFS Recursive Helper
void DFSUtil(struct Graph* graph, int vertex, bool visited[]) {
    visited[vertex] = true;
    printf("%d ", vertex);

    struct AdjListNode* temp = graph->adjLists[vertex];
    while (temp != NULL) {
        int connectedVertex = temp->dest;
        if (!visited[connectedVertex]) {
            DFSUtil(graph, connectedVertex, visited);
```

```
        }
        temp = temp->next;
    }
}

// DFS Traversal starting from startVertex
void DFS(struct Graph* graph, int startVertex) {
    bool* visited = (bool*)calloc(graph->V, sizeof(bool));

    printf("Depth First Search Traversal starting from vertex %d:\n  -> ", startVertex);
    DFSUtil(graph, startVertex, visited);
    printf("\n");

    free(visited);
}

void printComplexityAnalysis() {
    printf("\n================ COMPLEXITY ANALYSIS ================\n");
    printf(" 1. Time Complexity:\n");
    printf("    - Adjacency List: O(V + E)\n");
    printf("      Where V is the number of vertices and E is the number of edges.\n");
    printf("      Every vertex is visited once, and each edge is traversed once.\n");
    printf(" 2. Space Complexity:\n");
    printf("    - O(V) for the visited boolean array and function recursion stack\n");
    printf("      (in the worst case of a skewed/path graph, call stack depth is V).\n");
    printf("=====================================================\n");
}

void freeGraph(struct Graph* graph) {
    for (int i = 0; i < graph->V; i++) {
        struct AdjListNode* curr = graph->adjLists[i];
        while (curr != NULL) {
            struct AdjListNode* temp = curr;
            curr = curr->next;
            free(temp);
        }
    }
    free(graph->adjLists);
    free(graph);
}

int main() {
    printHeader();

    int V = 6;
    struct Graph* graph = createGraph(V);

    /*
          Graph Structure:
             0 --- 1
             |   / |
             |  /  |
             2 --- 3
             \    /
              4  5
    */
    addEdge(graph, 0, 1);
    addEdge(graph, 0, 2);
    addEdge(graph, 1, 2);
    addEdge(graph, 1, 3);
    addEdge(graph, 2, 3);
    addEdge(graph, 2, 4);
    addEdge(graph, 3, 5);

    printf("Graph with %d vertices constructed:\n", V);
    printf("Edges: (0-1), (0-2), (1-2), (1-3), (2-3), (2-4), (3-5)\n\n");
```

```
    DFS(graph, 0);

    printComplexityAnalysis();

    freeGraph(graph);
    return 0;
}
```

Figure 9.2: Terminal output for Depth First Search Traversal (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

Graph with 6 vertices constructed:
Edges: (0-1), (0-2), (1-2), (1-3), (2-3), (2-4), (3-5)

Depth First Search Traversal starting from vertex 0:
  -> 0 2 4 3 5 1

================ COMPLEXITY ANALYSIS ================
 1. Time Complexity:
    - Adjacency List: O(V + E)
      Where V is the number of vertices and E is the number of edges.
      Every vertex is visited once, and each edge is traversed once.
 2. Space Complexity:
    - O(V) for the visited boolean array and function recursion stack
      (in the worst case of a skewed/path graph, call stack depth is V).
==================================================
```

Program 9.3: Cycle Detection in Directed and Undirected Graphs (E9Q3.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

/* ============================================================
   PART 1: CYCLE DETECTION IN DIRECTED GRAPH
   ============================================================ */
#define MAX_V 20

struct DirectedGraph {
    int V;
    int adj[MAX_V][MAX_V];
};

struct DirectedGraph* createDirectedGraph(int V) {
    struct DirectedGraph* g = (struct DirectedGraph*)malloc(sizeof(struct DirectedGraph));
    g->V = V;
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
```

```
            g->adj[i][j] = 0;
        }
    }
    return g;
}

void addDirectedEdge(struct DirectedGraph* g, int u, int v) {
    g->adj[u][v] = 1;
}

bool isCyclicDirectedUtil(struct DirectedGraph* g, int v, bool visited[], bool recStack[]) {
    visited[v] = true;
    recStack[v] = true;

    for (int i = 0; i < g->V; i++) {
        if (g->adj[v][i]) {
            if (!visited[i] && isCyclicDirectedUtil(g, i, visited, recStack)) {
                return true;
            } else if (recStack[i]) {
                return true; // Back edge detected
            }
        }
    }

    recStack[v] = false; // Remove vertex from recursion stack
    return false;
}

bool isCyclicDirected(struct DirectedGraph* g) {
    bool visited[MAX_V] = {false};
    bool recStack[MAX_V] = {false};

    for (int i = 0; i < g->V; i++) {
        if (!visited[i]) {
            if (isCyclicDirectedUtil(g, i, visited, recStack)) {
                return true;
            }
        }
    }
    return false;
}

/* ============================================================
   PART 2: CYCLE DETECTION IN UNDIRECTED GRAPH
   ============================================================ */
struct UndirectedGraph {
    int V;
    int adj[MAX_V][MAX_V];
};

struct UndirectedGraph* createUndirectedGraph(int V) {
    struct UndirectedGraph* g = (struct UndirectedGraph*)malloc(sizeof(struct
UndirectedGraph));
    g->V = V;
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            g->adj[i][j] = 0;
        }
    }
    return g;
}

void addUndirectedEdge(struct UndirectedGraph* g, int u, int v) {
    g->adj[u][v] = 1;
    g->adj[v][u] = 1;
}
```

```
bool isCyclicUndirectedUtil(struct UndirectedGraph* g, int v, bool visited[], int parent) {
    visited[v] = true;

    for (int i = 0; i < g->V; i++) {
        if (g->adj[v][i]) {
            if (!visited[i]) {
                if (isCyclicUndirectedUtil(g, i, visited, v)) {
                    return true;
                }
            } else if (i != parent) {
                // Adjacent visited node is NOT parent -> cycle detected
                return true;
            }
        }
    }
    return false;
}

bool isCyclicUndirected(struct UndirectedGraph* g) {
    bool visited[MAX_V] = {false};

    for (int i = 0; i < g->V; i++) {
        if (!visited[i]) {
            if (isCyclicUndirectedUtil(g, i, visited, -1)) {
                return true;
            }
        }
    }
    return false;
}

int main() {
    printHeader();

    // 1. Directed Graph Cycle Test
    printf("============================================================\n");
    printf("       1. TESTING DIRECTED GRAPH CYCLE DETECTION            \n");
    printf("============================================================\n");
    struct DirectedGraph* dg1 = createDirectedGraph(4);
    addDirectedEdge(dg1, 0, 1);
    addDirectedEdge(dg1, 1, 2);
    addDirectedEdge(dg1, 2, 0); // Creates cycle: 0 -> 1 -> 2 -> 0
    addDirectedEdge(dg1, 2, 3);

    printf("Directed Graph 1 Edges: (0->1), (1->2), (2->0), (2->3)\n");
    printf("  >> Contains Cycle: %s\n\n", isCyclicDirected(dg1) ? "TRUE (Cycle Detected)" :
"FALSE (Acyclic)");

    struct DirectedGraph* dg2 = createDirectedGraph(3);
    addDirectedEdge(dg2, 0, 1);
    addDirectedEdge(dg2, 1, 2);
    printf("Directed Graph 2 (DAG) Edges: (0->1), (1->2)\n");
    printf("  >> Contains Cycle: %s\n", isCyclicDirected(dg2) ? "TRUE (Cycle Detected)" :
"FALSE (Acyclic)");

    // 2. Undirected Graph Cycle Test
    printf("\n============================================================\n");
    printf("      2. TESTING UNDIRECTED GRAPH CYCLE DETECTION           \n");
    printf("============================================================\n");
    struct UndirectedGraph* ug1 = createUndirectedGraph(4);
    addUndirectedEdge(ug1, 0, 1);
    addUndirectedEdge(ug1, 1, 2);
    addUndirectedEdge(ug1, 2, 3);
    addUndirectedEdge(ug1, 3, 0); // Creates cycle: 0-1-2-3-0
```

```
    printf("Undirected Graph 1 Edges: (0-1), (1-2), (2-3), (3-0)\n");
    printf("  >> Contains Cycle: %s\n\n", isCyclicUndirected(ug1) ? "TRUE (Cycle Detected)"
: "FALSE (Acyclic)");

    struct UndirectedGraph* ug2 = createUndirectedGraph(3);
    addUndirectedEdge(ug2, 0, 1);
    addUndirectedEdge(ug2, 1, 2);
    printf("Undirected Graph 2 (Tree) Edges: (0-1), (1-2)\n");
    printf("  >> Contains Cycle: %s\n", isCyclicUndirected(ug2) ? "TRUE (Cycle Detected)" :
"FALSE (Acyclic)");

    printf("============================================================\n");
    free(dg1);
    free(dg2);
    free(ug1);
    free(ug2);
    return 0;
}
```

Figure 9.3: Terminal output for Directed and Undirected Graph Cycle Detection (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

============================================================
       1. TESTING DIRECTED GRAPH CYCLE DETECTION
============================================================
Directed Graph 1 Edges: (0->1), (1->2), (2->0), (2->3)
  >> Contains Cycle: TRUE (Cycle Detected)

Directed Graph 2 (DAG) Edges: (0->1), (1->2)
  >> Contains Cycle: FALSE (Acyclic)

============================================================
      2. TESTING UNDIRECTED GRAPH CYCLE DETECTION
============================================================
Undirected Graph 1 Edges: (0-1), (1-2), (2-3), (3-0)
  >> Contains Cycle: TRUE (Cycle Detected)

Undirected Graph 2 (Tree) Edges: (0-1), (1-2)
  >> Contains Cycle: FALSE (Acyclic)
============================================================
```

Program 9.4: Kruskal's Minimum Spanning Tree Algorithm (E9Q4.c)

```
#include <stdio.h>
#include <stdlib.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}
```

```c
// Edge representation
struct Edge {
    int src, dest, weight;
};

// Graph representation
struct Graph {
    int V, E;
    struct Edge* edge;
};

struct Graph* createGraph(int V, int E) {
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->E = E;
    graph->edge = (struct Edge*)malloc(E * sizeof(struct Edge));
    return graph;
}

// Subset for Disjoint Set Union (DSU)
struct Subset {
    int parent;
    int rank;
};

// Find set of an element i (with path compression)
int find(struct Subset subsets[], int i) {
    if (subsets[i].parent != i) {
        subsets[i].parent = find(subsets, subsets[i].parent);
    }
    return subsets[i].parent;
}

// Union of two sets by rank
void Union(struct Subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);

    if (subsets[xroot].rank < subsets[yroot].rank) {
        subsets[xroot].parent = yroot;
    } else if (subsets[xroot].rank > subsets[yroot].rank) {
        subsets[yroot].parent = xroot;
    } else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}

// Compare function for qsort
int compareEdges(const void* a, const void* b) {
    struct Edge* a1 = (struct Edge*)a;
    struct Edge* b1 = (struct Edge*)b;
    return a1->weight - b1->weight;
}

// Kruskal's algorithm
void KruskalMST(struct Graph* graph) {
    int V = graph->V;
    struct Edge result[V]; // Stores the resultant MST
    int e = 0; // Index for result[]
    int i = 0; // Index for sorted edges

    // Step 1: Sort all edges in non-decreasing order of their weight
    qsort(graph->edge, graph->E, sizeof(graph->edge[0]), compareEdges);

    // Allocate memory for creating V subsets
```

```
    struct Subset* subsets = (struct Subset*)malloc(V * sizeof(struct Subset));
    for (int v = 0; v < V; v++) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
    }

    printf("--- Edges Sorted by Weight (Greedy Selection) ---\n");
    for (int k = 0; k < graph->E; k++) {
        printf("  Edge (%d - %d) with Weight = %d\n",
               graph->edge[k].src, graph->edge[k].dest, graph->edge[k].weight);
    }
    printf("\n");

    // Number of edges to be taken is equal to V - 1
    while (e < V - 1 && i < graph->E) {
        // Step 2: Pick the smallest edge
        struct Edge next_edge = graph->edge[i++];

        int x = find(subsets, next_edge.src);
        int y = find(subsets, next_edge.dest);

        // If including this edge doesn't cause cycle, include it in result
        if (x != y) {
            result[e++] = next_edge;
            Union(subsets, x, y);
        }
        // Else discard next_edge
    }

    // Print MST
    printf("============================================================\n");
    printf("          MINIMUM SPANNING TREE (MST) RESULT                \n");
    printf("============================================================\n");
    printf(" Edge       Weight\n");
    printf("--------------------\n");
    int minimumCost = 0;
    for (i = 0; i < e; i++) {
        printf(" %d -- %d      %d\n", result[i].src, result[i].dest, result[i].weight);
        minimumCost += result[i].weight;
    }
    printf("--------------------\n");
    printf(" Total Minimum Spanning Tree Cost: %d\n", minimumCost);
    printf("============================================================\n");

    free(subsets);
}

int main() {
    printHeader();

    /*
            Example Graph with 4 vertices and 5 edges:
                 0
               / | \
             10  6   5
             /   |    \
            1 -- 15 -- 2
             \         /
               4     3
                 \ /
                  3
    */
    int V = 4;
    int E = 5;
    struct Graph* graph = createGraph(V, E);
```

```
    // Edge 0-1
    graph->edge[0].src = 0;
    graph->edge[0].dest = 1;
    graph->edge[0].weight = 10;

    // Edge 0-2
    graph->edge[1].src = 0;
    graph->edge[1].dest = 2;
    graph->edge[1].weight = 6;

    // Edge 0-3
    graph->edge[2].src = 0;
    graph->edge[2].dest = 3;
    graph->edge[2].weight = 5;

    // Edge 1-3
    graph->edge[3].src = 1;
    graph->edge[3].dest = 3;
    graph->edge[3].weight = 15;

    // Edge 2-3
    graph->edge[4].src = 2;
    graph->edge[4].dest = 3;
    graph->edge[4].weight = 4;

    KruskalMST(graph);

    free(graph->edge);
    free(graph);
    return 0;
}
```

Figure 9.4: Terminal output for Kruskal's Minimum Spanning Tree Result (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Edges Sorted by Weight (Greedy Selection) ---
  Edge (2 - 3) with Weight = 4
  Edge (0 - 3) with Weight = 5
  Edge (0 - 2) with Weight = 6
  Edge (0 - 1) with Weight = 10
  Edge (1 - 3) with Weight = 15

============================================================
          MINIMUM SPANNING TREE (MST) RESULT
============================================================
 Edge       Weight
------------------------
 2 -- 3      4
 0 -- 3      5
 0 -- 1      10
------------------------
 Total Minimum Spanning Tree Cost: 19
============================================================
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• BFS (queue-based level order) and DFS (stack-based depth exploration) run in O(V + E) time with O(V) space.

• Directed graph cycle detection checks for back-edges to ancestors currently in the recursion call stack; undirected cycle detection verifies back-edges to already-visited non-parent nodes.

• Kruskal's MST algorithm sorts all edges in O(E log E) and greedily connects disjoint components using Union-Find with path compression in O(E alpha(V)) time.

EXPERIMENT NO. 10

Hashing (Direct Index Mapping Search in O(1) & Subset Verification)

1. Objective

To study hashing principles, direct index mapping (trivial hashing) supporting both positive and negative values in limited ranges [-MAX, +MAX] for O(1) search, and hash-table collision chaining for O(N + M) subset array verification.

2. Problem Statements / Program Definitions

1. Given a limited range array containing both positive and non-positive numbers, i.e., elements are in the range from -MAX to +MAX. Search if some number is present in the array or not in O(1) time.

2. Given two arrays: A and B. Find whether B is a subset of A or not using Hashing. Both the arrays are not in sorted order. Elements in both arrays are distinct.

3. Implementation & Output Snapshots

Program 10.1: O(1) Direct Index Mapping Search (E10Q1.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

#define MAX 1000

// 2D Hash table:
// has[X][0] = 1 if +X is present
// has[X][1] = 1 if -X is present
bool has[MAX + 1][2];

// Insert elements of array into the hash table in O(1) time each
void insert(int a[], int n) {
    for (int i = 0; i < n; i++) {
        if (abs(a[i]) > MAX) {
            printf("Warning: Element %d exceeds MAX limit (%d), ignored.\n", a[i], MAX);
            continue;
        }
        if (a[i] >= 0) {
            has[a[i]][0] = true;
        } else {
            has[abs(a[i])][1] = true;
        }
    }
}

// Search if X is present in O(1) time
bool search(int X) {
    if (abs(X) > MAX) {
        return false;
    }
    if (X >= 0) {
        return has[X][0];
    } else {
```

```
        return has[abs(X)][1];
    }
}

int main() {
    printHeader();

    // Example array from lab manual: contains negative and positive numbers
    int a[] = {-1, 9, -5, -8, -5, -2};
    int n = sizeof(a) / sizeof(a[0]);

    printf("Input Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");
    printf("Hash table constructed using 2D Direct Index Mapping (has[MAX+1][2]).\n\n");

    insert(a, n);

    // Test cases
    int testKeys[] = {-5, 9, -2, 7, -8, 15};
    int numTests = sizeof(testKeys) / sizeof(testKeys[0]);

    printf("--- Direct O(1) Search Results ---\n");
    for (int i = 0; i < numTests; i++) {
        int key = testKeys[i];
        if (search(key)) {
            printf("  Query: %3d -> PRESENT\n", key);
        } else {
            printf("  Query: %3d -> NOT PRESENT\n", key);
        }
    }

    // Interactive query
    printf("\n--- Interactive Query ---\n");
    int userQuery;
    printf("Enter a number to search in O(1) time: ");
    if (scanf("%d", &userQuery) == 1) {
        if (search(userQuery)) {
            printf("  Result: %d is PRESENT in the array.\n", userQuery);
        } else {
            printf("  Result: %d is NOT PRESENT in the array.\n", userQuery);
        }
    }

    printf("============================================================\n");
    return 0;
}
```

Figure 10.1: Terminal output for O(1) Direct Index Mapping with positive and negative keys (25BCP113)

```
==========================================================
  Name    : Ishan Patel
  Roll No: 25BCP113
==========================================================

Input Array: -1 9 -5 -8 -5 -2
Hash table constructed using 2D Direct Index Mapping (has[MAX+1][2]).

--- Direct O(1) Search Results ---
  Query:  -5 -> PRESENT
  Query:   9 -> PRESENT
  Query:  -2 -> PRESENT
  Query:   7 -> NOT PRESENT
  Query:  -8 -> PRESENT
  Query:  15 -> NOT PRESENT

--- Interactive Query ---
Enter a number to search in O(1) time: 10
  Result: 10 is NOT PRESENT in the array.
==========================================================
```

Program 10.2: Array Subset Verification using Hashing (E10Q2.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

void printHeader() {
    printf("==================================================\n");
    printf("  Name   : Ishan Patel                            \n");
    printf("  Roll No: 25BCP113                               \n");
    printf("==================================================\n\n");
}

#define HASH_TABLE_SIZE 101

// Hash Table Node for collision resolution (separate chaining)
struct HashNode {
    int key;
    struct HashNode* next;
};

struct HashTable {
    struct HashNode* table[HASH_TABLE_SIZE];
};

struct HashTable* createHashTable() {
    struct HashTable* ht = (struct HashTable*)malloc(sizeof(struct HashTable));
    for (int i = 0; i < HASH_TABLE_SIZE; i++) {
        ht->table[i] = NULL;
    }
    return ht;
}

// Hash Function (handles negative keys properly)
int hashFunction(int key) {
    int hash = key % HASH_TABLE_SIZE;
    if (hash < 0) {
        hash += HASH_TABLE_SIZE;
    }
```

```
    return hash;
}

// Insert key into Hash Table
void insertHash(struct HashTable* ht, int key) {
    int index = hashFunction(key);
    struct HashNode* newNode = (struct HashNode*)malloc(sizeof(struct HashNode));
    newNode->key = key;
    newNode->next = ht->table[index];
    ht->table[index] = newNode;
}

// Search key in Hash Table: returns true if found
bool searchHash(struct HashTable* ht, int key) {
    int index = hashFunction(key);
    struct HashNode* curr = ht->table[index];
    while (curr != NULL) {
        if (curr->key == key) {
            return true;
        }
        curr = curr->next;
    }
    return false;
}

void freeHashTable(struct HashTable* ht) {
    for (int i = 0; i < HASH_TABLE_SIZE; i++) {
        struct HashNode* curr = ht->table[i];
        while (curr != NULL) {
            struct HashNode* temp = curr;
            curr = curr->next;
            free(temp);
        }
    }
    free(ht);
}

// Function to check if array B is a subset of array A using Hashing
bool isSubset(int arrA[], int nA, int arrB[], int nB) {
    struct HashTable* ht = createHashTable();

    // Step 1: Insert all elements of array A into hash table
    for (int i = 0; i < nA; i++) {
        insertHash(ht, arrA[i]);
    }

    // Step 2: Search each element of array B in the hash table
    for (int i = 0; i < nB; i++) {
        if (!searchHash(ht, arrB[i])) {
            freeHashTable(ht);
            return false; // Found an element in B that is not in A
        }
    }

    freeHashTable(ht);
    return true; // All elements of B are present in A
}

void printArray(int arr[], int n) {
    printf("{ ");
    for (int i = 0; i < n; i++) {
        printf("%d%s", arr[i], (i == n - 1) ? "" : ", ");
    }
    printf(" }\n");
}
```

```c
int main() {
    printHeader();

    // Test Case 1: B is a subset of A
    int arrA1[] = {11, 1, 13, 21, 3, 7};
    int arrB1[] = {11, 3, 7, 1};
    int nA1 = sizeof(arrA1) / sizeof(arrA1[0]);
    int nB1 = sizeof(arrB1) / sizeof(arrB1[0]);

    printf("--- Test Case 1 ---\n");
    printf("Array A: "); printArray(arrA1, nA1);
    printf("Array B: "); printArray(arrB1, nB1);
    if (isSubset(arrA1, nA1, arrB1, nB1)) {
        printf(">> Result: Array B IS a subset of Array A.\n");
    } else {
        printf(">> Result: Array B is NOT a subset of Array A.\n");
    }

    // Test Case 2: B is not a subset of A
    int arrA2[] = {10, 5, 2, 23, 19};
    int arrB2[] = {19, 5, 3}; // 3 is not in A2
    int nA2 = sizeof(arrA2) / sizeof(arrA2[0]);
    int nB2 = sizeof(arrB2) / sizeof(arrB2[0]);

    printf("\n--- Test Case 2 ---\n");
    printf("Array A: "); printArray(arrA2, nA2);
    printf("Array B: "); printArray(arrB2, nB2);
    if (isSubset(arrA2, nA2, arrB2, nB2)) {
        printf(">> Result: Array B IS a subset of Array A.\n");
    } else {
        printf(">> Result: Array B is NOT a subset of Array A.\n");
    }

    printf("\n================ COMPLEXITY NOTE ================\n");
    printf(" Time Complexity : O(nA + nB) on average.\n");
    printf(" Space Complexity: O(nA) to store elements of array A.\n");
    printf("=================================================\n");

    return 0;
}
```

Figure 10.2: Terminal output for Subset Array Check using Hash Table Chaining (25BCP113)

```
==================================================
  Name   : Ishan Patel
  Roll No: 25BCP113
==================================================

--- Test Case 1 ---
Array A: { 11, 1, 13, 21, 3, 7 }
Array B: { 11, 3, 7, 1 }
>> Result: Array B IS a subset of Array A.

--- Test Case 2 ---
Array A: { 10, 5, 2, 23, 19 }
Array B: { 19, 5, 3 }
>> Result: Array B is NOT a subset of Array A.

================ COMPLEXITY NOTE ================
 Time Complexity : O(nA + nB) on average.
 Space Complexity: O(nA) to store elements of array A.
=================================================
```

4. Learning Outcome / Take Away

Key Takeaways & Complexity Analysis:

• Direct index mapping achieves instantaneous O(1) lookup by mapping integer keys directly to array indices, using a 2D sign array has[MAX+1][2] to support negative numbers.

• Hash Tables with separate chaining achieve O(1) average search and insert, allowing subset determination in O(N + M) time compared to O(N * M) brute-force nested iterations.